The Journal of Systems and Software 226 (2025) 112435

Contents lists available at ScienceDirect

The Journal of Systems & Software

journal homepage: www.elsevier.com/locate/jss

# BEACon-TD: Classifying Technical Debt and its types across diverse software projects issues using transformers☆

Karthik Shivashankar [a], Mili Orucevic [b], Maren Maritsdatter Kruke [b], Antonio Martini [a,*]

[a] University of Oslo, Norway
[b] Visma Software International, Norway

ARTICLE INFO

*Keywords:*
Technical Debt
AI
NLP
Software engineering

ABSTRACT

Technical Debt (TD) identification in software projects issues is crucial for maintaining code quality, reducing long-term maintenance costs, and improving overall project health. This study advances TD identification in issues tracker using transformer-based models, addressing the critical need for accurate and efficient TD identification in large-scale software development.

Our methodology employs multiple binary classifiers for TD and its type, combined through ensemble learning, to enhance accuracy and robustness in detecting various forms of TD. We train and evaluate these models on a comprehensive dataset from GitHub Archive Issues (2015–2024), supplemented with industrial data validation.

We demonstrate that in-project fine-tuned transformer models significantly outperform task-specific fine-tuned models in TD classification, highlighting the importance of project-specific context in accurate TD identification. Our research also reveals the superiority of specialized binary classifiers over multi-class models for TD and its type identification, enabling more targeted debt resolution strategies. A comparative analysis shows that the smaller DistilRoBERTa model is more effective than larger language models like GPTs for TD classification tasks, especially after fine-tuning, offering insights into efficient model selection for specific TD detection tasks.

The study also assesses generalization capabilities using metrics such as MCC, AUC ROC, Recall, and F1 score, focusing on model effectiveness, fine-tuning impact, and relative performance. By validating our approach on out-of-distribution and real-world industrial datasets, we ensure practical applicability, addressing the diverse nature of software projects.

This research significantly enhances TD detection and offers a more nuanced understanding of TD types, contributing to improved software maintenance strategies in both academic and industrial settings. The release of our curated dataset aims to stimulate further advancements in TD classification research, ultimately enhancing software project outcomes and development practices by enabling early TD identification and management.

## 1. Introduction

Technical Debt (TD) is the future cost of additional work in software development that arises from choosing quick, suboptimal solutions or from evolving requirements and technological advancements. Whether resulting from intentional decisions or external factors, TD necessitates careful management to maintain long-term project viability. Like financial debt, TD can accrue 'interest' over time, increasing complexity and maintenance costs (Besker et al., 2020; Holvitie et al., 2018).

TD can be both intentional and unintentional. Intentional TD occurs when developers consciously opt for a faster solution to meet short-term goals, understanding that they will address the deficiencies later (Pinna

☆ Editor: Raffaela Mirandola.

* Corresponding author.
*E-mail address:* antonima@ifi.uio.no (A. Martini).

https://doi.org/10.1016/j.jss.2025.112435
Received 30 August 2024; Received in revised form 19 December 2024; Accepted 7 February 2025
Available online 18 March 2025
0164-1212/© 2025 The Authors. Published by Elsevier Inc. This is an open access article under the CC BY license (http://creativecommons.org/licenses/by/4.0/).

et al., 2023). Unintentional TD, on the other hand, results from inadvertent mistakes, poor practices, or a lack of knowledge or insights during the development process (Faus et al., 2023; Fowler, 2009). TD manifests in various forms, including complex and hard-to-maintain code, lack of documentation, duplicated code, and outdated libraries or frameworks. Such debts can degrade software quality, increase maintenance costs, and reduce developer productivity. Moreover, excessive TD can lead to missed business opportunities and potentially result in a development standstill, causing companies to lose clients. Consequently, effective management and timely resolution of TD are critical to maintaining the health, competitiveness, and sustainability of software projects (Besker et al., 2020; Spínola et al., 2019).

With the increasing software complexity and rapid development cycles, traditional methods of identifying and managing TD — such as manual code reviews and static analysis tools; often prove insufficient. These methods can be labor-intensive, prone to human error, and may not scale well with the size and complexity of modern software systems (Avgeriou et al., 2023). This necessitates exploring automated techniques to detect and classify TD issues from project or source control management systems like Github or Jira, enabling timely interventions and proactive assessment of TD and its interest needed to be paid (Skryseth et al., 2023). Automated TD tracking offers several advantages. It reduces the workload associated with manual tracking in backlogs, which teams often resist due to its time-consuming nature. Additionally, it can help identify TD that might otherwise go unnoticed, educating team members on potential issues. Furthermore, it enables analytics on the quantity and types of TD accumulating, facilitating more informed decision-making and proactive management of technical debt (Ren et al., 2022b).

Recent advancements in Deep Learning, particularly transformer-based models like BERT and GPT (Devlin et al., 2018; Zhuang et al., 2021; Radford et al., 2019), have shown promise in complex natural language processing tasks. While these models have been applied to various software engineering problems, their full potential in TD classification remains unexplored. Previous studies have made significant strides in TD detection and classification. DebtHunter (Liu et al., 2021) combined code metrics and textual features with ensemble learning, while Huang et al.'s (Self Admitted Technical Debt) SATD Detector (Huang et al., 2018) used NLP techniques and machine learning classifiers. Ren et al. (2022b,a) explored SATD detection in issue tracking systems using text classification and semi-supervised learning.

Li et al. (2023) proposed an MT-Text-CNN approach for multi-source SATD detection. However, these approaches have limitations. Rule-based and static analysis tools often struggle with context and nuance (Liu et al., 2018; Lefever et al., 2021). Traditional machine learning models, while effective, may not capture the full complexity of natural language in TD descriptions (Simon et al., 2024; Cruz et al., 2020).

Even advanced approaches like (Li et al., 2023) MT-Text-CNN achieved an average F1-score of only around 0.61 across different TD sources. The gap lies in leveraging the full potential of transformer models for TD classification and testing models generalization capabilities in Out of Distribution (OOD) dataset and industrial data . These transformer models excel at capturing context and nuances in natural language (von der Mosel et al., 2023; Maldonado et al., 2017), which is crucial for interpreting ambiguous TD descriptions across various sources in issue tracking software.

Understanding and classifying different types of TD is crucial for effective management and mitigation strategies. However, accurately identifying and categorizing various TD types (Alves et al., 2014) presents a significant challenge in software engineering research and practice.

Our study aims to explore these challenges by comparing different classification approaches and developing a ensemble method for TD and its type identification. We focus on the 13 types of TD identified in the ontology by Alves et al. (2014), including architectural, test, build, and code debt, among others. By leveraging a comprehensive dataset of GitHub issues from 2015 to 2024, we seek to develop and evaluate models that can accurately classify TD types with high precision and recall.

This research contributes to the field by addressing the complexities of TD classification and proposing solutions to enhance the accuracy and reliability of TD type identification in software projects.

A critical aspect of our research involves evaluating the models' ability to generalize to unseen data. This is essential for ensuring the practical applicability of our TD classification approach in real-world scenarios. To achieve this, we assess the models' performance on an out-of-distribution (OOD) dataset (Hu and Khan, 2021), which includes projects not part of our training dataset. The use of OOD data provides a more rigorous test of our models' robustness and effectiveness in diverse, real-world software development environments.

This study seeks to answer the following key research questions:

- **RQ1:** How effective are transformer-based models in classifying TD issues?
- **RQ1.1:** Does fine-tuning TD models on project-specific data improve their performance?
- **RQ2:** How does the performance of LLM-like the GPT model compare to the DistilRoBERTa models in TD classification?
- **RQ2.1:** Is task specific fine-tuning of LLM like GPT for TD more effective than finetuned DistilRoBERTa TD classification?
- **RQ3:** How effective are expert ensemble of binary classifiers in classifying different types of issues compared to multi-class model?
- **RQ3.1**: How does the performance of LLM-like the GPT model compare to the DistilRoBERTa Model on different issues types?

Rationale for RQ1 and RQ1.1: Evaluating transformer models' effectiveness in TD classification is crucial for advancing automated detection. Also, to evaluate how fine-tuning may enhance performance by adapting to project-specific nuances and context.

Rationale for RQ2 and RQ2.1: Comparing GPT and DistilRoBERTa will provide insights into the balance between model type and size and classification performance, guiding future research and implementations.

Rationale for RQ3 and RQ3.1: Comparing an ensemble of multiple binary classifiers to a multi-class approach will determine the most effective strategy for handling diverse TD types. Assessing GPT and DistilRoBERTa across TD types will reveal their strengths in various aspects of technical debt and its type.

This research makes several significant contributions to the field of software engineering:

- **Empirical Assessment of Transformer-Based Models to classify TD:** This study comprehensively evaluates transformer-based models, specifically GPT and DistilRoBERTa, for TD classification. It examines the impact of fine-tuning on their performance, offering insights into their effectiveness and potential for practical application.
- **Specialized Binary Classifiers for TD Types:** We validate the use of multiple ensemble binary classifiers for identifying specific TD types (e.g., architectural, testing, build). This approach is compared with a multi-class classification approach, highlighting the advantages of using multiple ensemble binary models for precise and robust TD type identification.
- **Generalization to Out-of-Distribution Data:** The study empirically evaluates the models' ability to generalize to out-of-distribution (OOD) data, addressing a critical challenge for the practical adoption of TD classification tools in diverse software projects.
- **Publicly Available Dataset:** We will release our curated dataset of technical debt and 13 different types of TD category issues sourced from GitHub Archive Issues events from 2015 to 2024. This dataset will support further research and development in TD classification, fostering advancements in software maintenance.

### 1.1. How is this study different from our previous work

The study represents a significant advancement over earlier work (Skryseth et al., 2023) in TD detection and classification. While previous studies focused on binary TD classification, In this work, we introduce a comprehensive approach that can classifies 13 distinct types of TD. This provides a more nuanced understanding of TD in software projects, enabling more targeted remediation strategies. Another key contribution in this study, is that we combines multiple binary classifiers using a simple ensemble approach, each specialized in detecting a specific type of TD. This approach significantly improves classification accuracy compared to multi-class models.

Additionally, it offers the first comprehensive comparison between smaller BERT based models like DistilRoBERTa and to GPT models for TD classification, providing valuable insights into model selection for TD detection tasks.

In this study, we have also includes validation on a large-scale industrial dataset from a multinational software company. This real-world validation enhances the practical applicability of the approach. Furthermore, in this paper, we evaluates the model's ability to detect TD in future data, simulating real-world scenarios where models must predict TD in ongoing projects. This temporal analysis provides insights into the long-term applicability of the approach, an aspect often overlooked in earlier research.

Finally, this study contributes a comprehensive, curated dataset of 13 TD types extracted from GitHub issues (2015–2024). This dataset will facilitate further research in automated TD management and enable comparative studies in the field, addressing a gap in available resources for TD research.

## 2. Background and related works

### 2.1. Transformer models

Transformer architectures have revolutionized the landscape of natural language processing (NLP), introducing a paradigm shift in how machines interpret and process human language. These models capture contextual relationships between words in a sentence, irrespective of their sequential order. At the heart of transformers lies the attention mechanism, which enables the model to weigh the importance of different input parts when producing an output (Vaswani et al., 2017). This approach allows transformers to establish global dependencies within the text, leading to more nuanced and context-aware language understanding

Transformer models have a set performance benchmark in text classification, where the goal is to assign predefined categories to textual data. The Hugging Face (HF) ecosystem has democratized the implementation of these sophisticated models by providing a user-friendly interface and a comprehensive collection of pre-trained architectures. HF offers APIs for a wide range of transformer variants, including BERT (Devlin et al., 2018), GPT (Brown et al., 2020), and RoBERTa (Zhuang et al., 2021), enabling researchers to fine-tune these models on domain-specific datasets (Wolf et al., 2019). This capability has revolutionized the field, allowing practitioners to achieve state-of-the-art results even with limited labeled data.

### 2.2. Comparison of LLMs and BERT-based models

Large Language Models (LLMs) like GPT-3 (Brown et al., 2020) and GPT-4 have revolutionized NLP by leveraging vast amounts of training data to generate human-like text and perform a wide range of language tasks. These models are particularly known for their generative capabilities, making them versatile tools for various applications, including text classification. On the other hand, models like DistilRoBERTa, a distilled version of RoBERTa, offer a more efficient alternative with reduced computational requirements while retaining a significant portion of the original model's performance (Sanh et al., 2019).

Several factors come into play when comparing the performance of LLMs like GPT 4 and DistilRoBERTa in TD classification. With their extensive training on diverse datasets, LLMs can handle complex language tasks and generate contextually relevant text. This makes them suitable for tasks that require a deep understanding of context and semantics. However, their large size and computational demands can be a drawback in resource-constrained environments (Kacprzak, 2024).

DistilRoBERTa, with its optimized architecture, offers a balance between performance and efficiency. It retains approximately 97% of RoBERTa's performance while being faster and more resource-efficient. This makes it a viable option for TD classification tasks where computational resources are limited. Studies have shown that fine-tuning DistilRoBERTa on domain-specific data can enhance its performance, making it comparable to larger models like GPT-4 in certain tasks (Sanh et al., 2019).

Both LLMs and DistilRoBERTa benefit from domain-specific training in terms of fine-tuning. Fine-tuning involves adapting the pre-trained model to the specific characteristics of the target domain, which can significantly improve classification accuracy (Bilal and Almazroi, 2023). However, the choice between these models ultimately depends on the specific requirements of the task, including the available computational resources and the complexity of the classification problem.

### 2.3. Related works

The impact of Technical Debt (TD) on software quality and maintenance costs has led researchers to explore increasingly sophisticated automated detection and management methods. TD classification and detection are crucial for maintaining software quality and managing project risks. Various methods have been employed, each with strengths and weaknesses.

Rule-based approaches utilize predefined rules or patterns to identify potential technical debt, often using pattern matching to detect Self-Admitted Technical Debt (SATD) in source code comments. While these methods can achieve high precision, they tend to have low recall, potentially missing many instances of debt (Liu et al., 2018). Static analysis tools like SonarQube, Structure 101, and DV8 analyze code without execution to assess various code metrics, including complexity, size, and code smells, flagging problematic areas. However, there is a lack of consensus on what constitutes technical debt, leading to varied results (Lefever et al., 2021).

Traditional machine learning models like SVM, Logistic Regression, and Random Forest, often used with feature extraction techniques like TF–IDF and Count Vectorizer, transform code comments or issue tracker data into a suitable format for ML algorithms. These models can provide good performance when optimized and used with appropriate features (Simon et al., 2024; Cruz et al., 2020).

Deep learning models, particularly BERT and other transformer-based models, excel at handling large datasets and capturing complex patterns, making them particularly effective in handling natural language nuances in code comments and issue trackers. BERT has demonstrated high F1 scores in detecting various types of technical debt in R package peer-review comments (Khan and Uddin, 2022).

Hybrid approaches, combining traditional methods with ML models, can further enhance detection. Integrating rule-based systems with ML models can improve recall, while using static analysis tools for preprocessing can improve feature extraction and model performance (Sridharan et al., 2021). The choice of technical debt detection method depends on project-specific needs and the nature of the codebase. Recent advancements in TD detection include DebtHunter by Liu et al. a machine learning-based approach combining code metrics and textual features with ensemble learning techniques to enhance detection accuracy (Liu et al., 2021).

Huang et al. developed SATD Detector, a text-mining-based tool using NLP techniques and machine learning classifiers to identify SATD

in source code comments (Huang et al., 2018). Ren et al. explored SATD detection in issue tracking systems using various text classification techniques, and to tackle limited labeled data, they proposed DebtFree, a semi-supervised learning approach to minimize labeling costs while maintaining high accuracy (Ren et al., 2022b,a).

Flisar and Podgorelec investigated deep learning methods for SATD detection, comparing various neural network architectures to traditional machine learning approaches (Flisar and Podgorelec, 2021). While their work provided valuable insights into the potential of deep learning for TD detection, our study goes further by not only using state-of-the-art transformer models but also comparing their performance with GPT models, offering a more comprehensive evaluation of current AI capabilities in TD classification.

Tan et al. proposed an ensemble learning approach, combining multiple classifiers to improve SATD detection accuracy and robustness (Tan et al., 2021). Our work builds upon this concept but takes it further by developing an ensemble of binary classifiers for multiple TD types, significantly improving classification accuracy compared to multi-class models. This approach allows for more nuanced and accurate TD detection across a broader range of TD categories.

Li et al. (2023) proposed an integrated approach using MT-TextCNN to identify four types of SATD across four different sources (source code comments, issue trackers, pull requests, and commit messages). This represented a significant step towards multi-source TD detection, achieving an average F1-score of 0.61. However, this performance still left room for improvement, particularly in handling the nuances of natural language across different TD sources. In contrast, our work significantly expands on this by classifying 13 distinct TD types and achieving higher F1-scores through the use of advanced transformer models and ensemble techniques.

Maldonado et al. utilized NLP techniques to automatically detect SATD, achieving high precision and recall in identifying design and requirement debt (Maldonado et al., 2017). Building on this foundation, Ren et al. developed a convolutional neural network (CNN) based approach, which outperformed traditional machine learning methods in accuracy and F1-score (Ren et al., 2019). To enhance the interpretability of SATD detection models, Huang et al. proposed a graph neural network (GNN) based approach, providing not only high accuracy but also explanations for predictions (Huang et al., 2022). While this approach offers valuable insights into model decisions, our work focuses on improving overall classification accuracy and generalizability across multiple TD types. Future work could potentially integrate interpretability techniques with our high-performing models to combine accuracy with explainability.

Recognizing the unique manifestations of TD in machine learning (ML) systems, Sculley et al. introduced the concept of "hidden technical debt in machine learning systems", identifying ML-specific debt types and their implications (Sculley et al., 2015). Guo et al. further addressed these challenges through an empirical study on refactoring and TD in ML systems, offering insights into the unique characteristics of TD in ML code (Guo et al., 2021).

Recent studies have further expanded our understanding of TD. Recupito et al. (2024) shed light on the challenges of AI technical debt, exploring its prevalence, severity, and impact on AI-enabled systems, as well as the strategies practitioners employ to detect and mitigate it (Recupito et al., 2024). Tsoukalas et al. (2024) propose the use of explainable AI models to interpret the factors contributing to high technical debt risk, enhancing transparency and trust in technical debt identification (Tsoukalas et al., 2024).

Sas and Avgeriou (2023) introduce a novel approach to estimate architectural technical debt principal based on machine learning and architectural smells, addressing shortcomings of previous methods and aiming for a more accurate and automated assessment of technical debt (Sas and Avgeriou, 2023). While their work focuses specifically on architectural TD, our approach provides a more comprehensive view by classifying multiple types of TD, including but not limited to architectural debt.

Our previous study (Skryseth et al., 2023) demonstrated that transformer-based models, like DeBERTa (He et al., 2021), exhibit exceptional performance in TD classification tasks. Building on this foundation, our current work significantly extends the scope and applicability of TD detection. We now classify 13 distinct TD types, offering a more comprehensive understanding of TD in software projects. Our study is the first to comprehensively compare multi-class and binary classification approaches for TD types, as well as evaluate the performance of transformer models like DistilRoBERTa and GPTs like GPT 3.5 turbo and 4o in this context.

Our current work not only builds upon these existing studies but also significantly advances the field by introducing novel contributions. We validate our models on a real-world dataset from a multinational software company, demonstrating their practical applicability and generalizability. This study represents a significant advancement over earlier work (Skryseth et al., 2023) in TD detection and classification. While previous studies focused on binary TD classification, we introduce a comprehensive approach that can classify 13 distinct types of TD, providing a more nuanced understanding of TD in software projects and enabling more targeted remediation strategies.

A key contribution in this study is the combination of multiple binary classifiers using a simple ensemble approach, each specialized in detecting a specific type of TD. This approach significantly improves classification accuracy compared to multi-class models. Additionally, we offer the first comprehensive comparison between smaller BERT-based models like DistilRoBERTa and GPT models for TD classification, providing valuable insights into model selection for TD detection tasks.

## 3. Methodology

The methodology for this research was designed to empirically evaluate the effectiveness of transformer-based models in classifying TD and its various types using issues extracted from GitHub. Our approach encompasses several stages as shown in Fig. 1: data mining, dataset processing and cleaning, model training, and model evaluation, including tests on out-of-distribution (OOD) datasets. Each stage is critical for ensuring the reliability and validity of the findings.

We have made the replication package publicly available at **Zenodo**.[1] This package includes all the code for training and evaluating the DistilRoBERTa and GPT models on various metrics. Additionally, we have made all datasets used in this paper available within this replication package except private project dataset from Visma.

### 3.1. Data mining

The initial stage of our methodology involved accumulating a large dataset of software development issues from GitHub, specifically targeting those related to TD and its 13 types. We utilized the GitHub Archive (GHArchive) (GHArchive, 2024), a project that records the public GitHub timeline, logs, and events. Data was mined from January 1, 2015, to May 25, 2024, to cover various software projects and incorporate various issue events.

In our study, we employed a carefully crafted regular expression (regex) pattern to identify TD related issues from GitHub Archive data. The regex, designed to be case-insensitive, targeted variations of the term "Technical debt" and its abbreviations in issue labels. Our pattern;

```
r'(?i)\b(T(echnical[-_\s]?|ech[-_\s]?)?D(ebt|D)
    |\b(TD|td)\b|debt)\b'
```

[1] https://zenodo.org/doi/10.5281/zenodo.12517204.

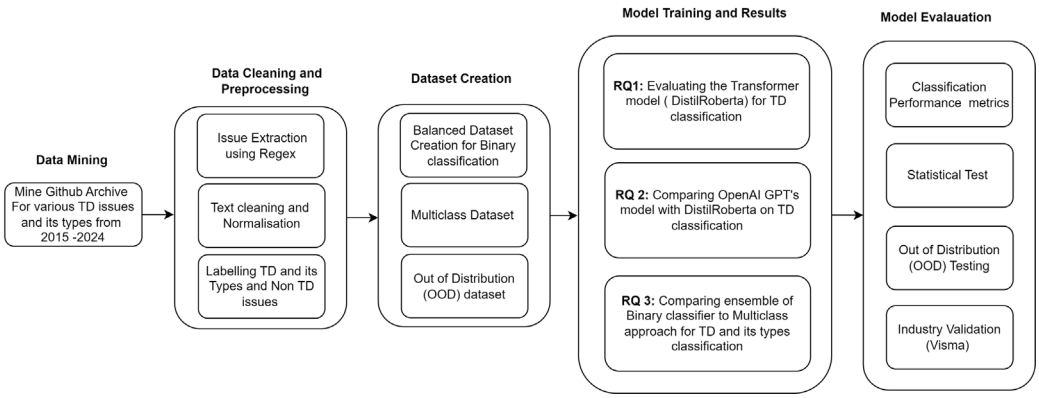


**Fig. 1.** High level overview of methodology and process.

effectively captured diverse representations such as "Technical debt", "Tech_debt", "TD", and standalone "debt" mentions from the label field in the issue event from GHarchive. This approach ensured comprehensive coverage of TD-related discussions while minimizing false positives. The pattern's flexibility allowed for various word separators and abbreviations commonly used by developers, enhancing our data collection's accuracy and completeness.

To identify issues related to specific types of TD (Alves et al., 2014), we employed a comprehensive regex pattern. The pattern was designed to capture 13 representations of TD types in a case-insensitive format within the 'labels' field of GitHub Issue events. Our regex pattern for issue types is as follows:

```
r'(?i)\b(architect(ure|ural)?|build|code|defect|
    design|doc(umentation)?|infrastructure|
    people|process|requirement|service|test(ing)
    ?|automation)\b'
```

The regex uses word boundaries (\b) to ensure accurate matching and includes common variations (e.g., 'doc' for 'documentation', 'testing' for 'test'). This approach allows for a nuanced classification of TD types in software projects, enabling a more detailed analysis of TD distribution and impact.

To ensure the effectiveness of our regex patterns, all authors independently reviewed 300 random samples of extracted issues. The validation focused on verifying whether the matched labels accurately represented technical debt and its types. This involved a manual review of a subset of the matched labels, allowing us to verify that they accurately represented technical debt and its various types. Any ambiguous cases were documented and the regex patterns were iteratively refined to improve precision. For instance, we checked for cases where "debt" might refer to financial debt. During our manual validation process, we found no instances where financial debt was mistakenly included. Through this examination, we were able to confirm that our patterns were successfully capturing relevant issues while minimizing false positives.

In addition to identifying issues related to TD and its specific types, we established a ground truth test dataset for our classification model evaluation. This dataset was created by selecting issues that explicitly contained both TD indicators with labels like "tech-debt", "debt" and "td" and specific TD-type labels like "architecture", "build", "test" and other TD types (Alves et al., 2014) that were unambiguously labeled by the project developers. These entries containing both these labels were removed from all our Datasets to avoid data leakage. This rigorous selection process provided a high-quality dataset for training and evaluating our TD classification models, enhancing the reliability and accuracy of our study results.

### 3.2. Dataset processing and cleaning

Once the data was accumulated, it underwent extensive preprocessing and cleaning to prepare it for effective model training. The raw data from GHArchive includes a lot of noise — duplicates, non-English text, irrelevant metadata, URLs, emojis, and special characters — that can obscure the underlying patterns related to TD.

The dataset underwent a comprehensive preprocessing phase to prepare it for model training. Raw data from GHArchive was cleaned using a systematic text preprocessing function in Python, which executed sequential transformations to optimize the text data quality.

First, all text was converted to lowercase to ensure consistency across the dataset. The function then systematically removed text enclosed in square brackets using regular expressions (`re.sub('\[.*?\]', ' ', text)`). To reduce noise, all URLs beginning with 'http://', 'https://', or 'www.' were eliminated using pattern matching (`re.sub('https?://\S+|www\.\S+', ' ', text)`).

HTML tags and special markup were stripped from the text using the pattern `<.?>+`. All punctuation marks were removed using the Python string.punctuation library, replacing them with spaces. The function also handled different types of line breaks by removing both Unix-style (`\n`) and Windows-style (`\r`) line endings. Finally, any words containing numerical digits were removed using regular expression pattern matching (`\w\d\w*`)

Therefore, the dataset was cleaned through several steps:

- **Duplicate Removal**: Ensuring that each issue is unique to prevent bias in model training.
- **Text Normalization**: Converting all text to lowercase to maintain consistency across the dataset.
- **Noise Reduction**: We removed links, emojis, and special characters that did not contribute to understanding the context of TD.
- **Content Filtering**: Eliminating issues with very few characters likely does not contain enough classification information.

**Text Preprocessing Example**

**Before Processing:**

```
[BUG] User Authentication Error #123
https://github.com/repo/issues/123

Steps to reproduce:
1. Navigate to www.example.com/login
2. Enter credentials
<strong>Error occurs</strong> on Chrome v89!

Related to PR [#456]
```

**After Processing:**

```
user authentication error
steps to reproduce
navigate to
enter credentials
error occurs on chrome
related to pr
```

Our study focused on 13 distinct categories of TD identified in the ontology by Alves et al.: Architecture, Code, Defect, People, Automation, Build, Process, Design, Requirement, Infrastructure, Test, Service, and Documentation (Alves et al., 2014) and TD itself.

We created separate datasets for each category, with the number of instances varying significantly across categories, as shown in Table 1. To ensure balanced datasets for robust model training, we implemented a data cleaning process. For each category, we created equal proportions of positive and negative labels, dividing the total instances equally between the two classes.

Our dataset comprises self-admitted technical debt (SATD) instances, where all labels were explicitly assigned by developers during the normal software development process on GitHub. These labels represent developers' conscious acknowledgment and documentation of technical debt in their projects. The labels (e.g., 'technical-debt', 'architecture-debt', 'code-debt') were not assigned through manual annotation or automated classification but were original tags created and applied by the development teams themselves when creating or managing issues.

For instance, when a developer creates an issue titled 'Refactor authentication module' and describes it as "The current implementation is difficult to maintain...", they explicitly tag it with labels such as ['technical-debt', 'refactoring']. This self-admission approach ensures that our dataset reflects genuine instances of TD recognition by practitioners rather than third-party interpretations. The labels serve as reliable indicators of TD since they represent developers' direct acknowledgment of technical compromises in their codebase.

To maintain dataset quality, we only considered issues where developers had explicitly tagged them with TD-related labels corresponding to our 13 categories (Architecture, Code, Defect, etc.). For the negative class, we randomly sampled from issues that developers had not tagged with any TD-related labels, ensuring that these issues were from the same projects and time periods as our positive samples to maintain consistency and relevance.

Sridharan et al. demonstrated the importance of data balance in SATD detection tasks, showing that imbalanced datasets can lead to biased models and unreliable predictions. This approach helps to mitigate the risk of model bias. It improves the overall reliability of our classification results across various TD types, extending the principles of balanced data from SATD detection to broader TD classification tasks. Sridharan et al. (2021).

**Table 1**
Total number of instances for different categories.

| Category | #Instances | Category | #Instances |
|---|---|---|---|
| Architecture | 20 466 | Design | 276 320 |
| Code | 249 934 | Requirement | 26 438 |
| Defect | 156 356 | Infrastructure | 59 546 |
| People | 2020 | Test | 608 634 |
| Automation | 13 356 | Service | 130 622 |
| Build | 191 780 | Documentation | 735 926 |
| Process | 37 532 | Technical Debt | 108 653 |

We created an out-of-distribution (OOD) dataset for robust evaluation for each category (Hu and Khan, 2021; Ovadia et al., 2019). We identified repositories with the highest number of issues for each type and withheld these from the training and testing datasets. This approach allowed us to assess the models' generalization capabilities across different project contexts.

The dataset splitting process was implemented using scikit-learn's function with a fixed random seed for reproducibility. The 85/15 ratio for training and testing sets was chosen after careful consideration of dataset sizes and common machine learning practices. The splitting was performed using stratified sampling to ensure that the proportion of samples for each class was preserved in both training and testing sets.

This random yet stratified approach maintained the balanced distribution of positive and negative labels in both subsets while avoiding any selection bias. For instance, if a TD category had 1000 total instances (500 positive, 500 negative), the training set would contain approximately 850 instances (425 positive, 425 negative), and the test set would contain approximately 150 instances (75 positive, 75 negative), all selected randomly while maintaining class proportions.

For the multiclass classification task, we adopted a different approach due to the inherent imbalance in TD type occurrences. We used only the positive labels from each category and employed stratified K-fold cross-validation for model training (Szeghalmy and Fazekas, 2023). We maintained the 85/15 train–test split and also created an OOD dataset for the multiclass model to ensure a comprehensive evaluation.

This methodology enabled us to develop and rigorously test both binary and multiclass models for TD classification, providing a thorough analysis of TD detection and categorization in diverse software development scenarios.

### 3.3. Model training

The training of the models was conducted using a carefully considered approach to ensure robustness and reliability. Binary classification models were trained on a balanced dataset to prevent any bias towards the more frequently occurring class (Sridharan et al., 2021). We employed 5-fold cross-validation over five epochs, a standard research practice that balances adequate model exposure to the data (Syed et al., 2020; Raj et al., 2021).

To achieve higher precision, recall and accuracy in classifying types of TD, we trained individual binary classifiers for each specific TD type (e.g., "test" vs. "not test", "architectural" vs. "not architectural", "code" vs. "not code"). This method employs an ensemble learning technique, combining multiple models to improve predictive performance by leveraging their strengths to enhance robustness and accuracy. Tan et al. (2021) demonstrated the effectiveness of ensemble learning techniques in improving predictive performance for Self-Admitted Technical Debt (SATD) detection tasks. They suggested that combining multiple models can enhance robustness and accuracy by leveraging the strengths of individual classifiers. Following their recommendations, we adopted an ensemble learning approach in our study for TD classification.

Our simple rule-based ensemble method integrates predictions from Classifier A and Classifier B. The classification process involves a two-stage assessment. Initially, Classifier A determines whether an issue

constitutes TD. Subsequently, if Classifier A flags the issue as TD, each specialized Classifier B evaluates it for its specific TD type (e.g., 'architecture', 'test'). An issue is only categorized as a specific TD type when both Classifier A and the corresponding specialist Classifier B concur. This cautious methodology minimizes false positives, enhancing the accuracy of our classifications. An instance is labeled as Architectural Technical Debt only if both classifiers produce positive results, indicating the presence of both architectural and technical debt.

This simple rule-based ensemble demonstrates the effectiveness of combining a general TD classifier with specialist classifiers we call this approach and model as **BEACon-TD** (**B**inary **E**nsemble for **A**utomated **C**lassificati**on** of **T**echnical **D**ebt), which we will discuss later in Results section. More complex ensemble techniques, such as weighted voting or stacking, could be explored by practitioners to further refine the classification process based on their specific needs and the relative importance of different TD types in their projects. By using this straightforward ensemble approach, we achieves improved accuracy in TD classification while maintaining simplicity and interpretability. This method allows for easy integration into existing software development workflows and can be readily adapted or extended by practitioners.

For our multiclass classification models, we utilized Stratified 5-fold cross-validation over five epochs to address the imbalanced nature of different TD types' occurrences. This method ensures that each fold maintains the same proportion of classes as in the full dataset, providing a representative sample for model training and evaluation (Szeghalmy and Fazekas, 2023). We incorporated class weights in computing the loss function to mitigate the impact of class imbalance in the multiclass classifier model. These weights were assigned to different classes based on their frequency in the dataset, giving more importance to underrepresented classes during training. This approach helps the model learn effectively from all classes, regardless of their prevalence in the dataset (Younes and Mathiak, 2022). By combining Stratified K-fold cross-validation with weighted loss computation, we aimed to develop robust multiclass models capable of accurately classifying various TD types despite their uneven distribution in real-world software projects.

*Hyperparameter settings.* All the models used in this study was fine-tuned on text data with a maximum sequence length. The training process employs a 5-fold cross-validation strategy with 5 epochs per fold. Key hyperparameters include: Batch sizes: 32 for training, 64 for evaluation; Learning rate scheduler: Cosine with 500 warmup steps; Weight decay: 0.01; Loss function: CrossEntropyLoss.

The training process utilizes FP16 for efficiency and implements gradient accumulation over 2 steps. The best model is saved after each epoch, with only the latest version retained.

Data processing involves tokenization with max-length padding and truncation. This configuration aims to create a robust and efficient model for text classification tasks, with particular attention to handling class imbalance and optimizing computational resources.

Expert binary classifiers focus on identifying specific characteristics of their assigned type and enhance robustness and accuracy when combined with the general TD classifier. Furthermore, binary classifiers simplify the classification task to a binary decision, which is less sensitive to data imbalance than a MultiClass approach, ensuring accurate identification and management of even less frequently tagged TD types.

### 3.4. Model evaluation and testing with OOD dataset

The final stage of our methodology involved rigorous evaluation and testing of the trained models. We assessed model performance using standard metrics such as accuracy, precision, recall, Matthews's correlation coefficient (MCC), AUC (Area Under The Curve) ROC (Receiver Operating Characteristics), and F1-score on a reserved test set. Precision measures the percentage of TD issues that have been correctly identified (true positives) out of all predictions. The Recall metric, which calculates the proportion of correctly identified TD issues (true positives) out of all actual TD issues (true positives + false negatives), holds particular importance in our study. Firstly, many software repositories contain only partially tagged TD issues. This means that the negative class in our dataset might inadvertently include some positive instances that were not explicitly labeled as TD. As a result, the precision metric, which relies on the accuracy of both positive and negative labels, cannot be considered entirely reliable in this scenario. In contrast, the Recall metric focuses solely on the correct identification of known TD issues, making it a more dependable measure of our model's performance. It assesses how effectively our approach captures all instances of TD, which is critical for comprehensive TD management.

The F1-score combines both precision and recall through a harmonic mean. The AUC is a performance measurement for classification problems at various threshold settings. It is the area under the Receiver Operating Characteristic (ROC) curve, which plots the true positive rate (TPR) against the false positive rate (FPR) at different threshold levels. The AUC value ranges from 0 to 1. An AUC of 0.5 suggests no discrimination (i.e., random guessing), while an AUC of 1 indicates perfect discrimination.

We also emphasize Matthews's Correlation Coefficient (MCC), which is widely recognized as a more statistically reliable metric (Chicco and Jurman, 2020-01-02) for binary classification.

$$MCC = \frac{TP \cdot TN - FP \cdot FN}{\sqrt{(TP + FP) \cdot (TP + FN) \cdot (TN + FP) \cdot (TN + FN)}}$$

MCC produces a more informative and truthful score in evaluating classifications than accuracy and F1 score. It produces a high score only if the prediction obtained good results in all four confusion matrix categories (true positives (TP), false negatives (FN), true negatives (TN), and false positives (FP)), proportionally both to the size of positive and negative elements in the dataset. MCC values range between 1 to −1, and a value greater than 0 indicates that the classifier is better than a random flip of a fair coin, whilst 0 means it is no better.

In addition, all models were also tested against an out-of-distribution (OOD) dataset, which consisted of data from projects not included in the training dataset. This step is crucial to determining the models' generalization capabilities to new, unseen data contexts.

Furthermore, we also perform comparative analysis, including evaluating the performance of BERT-family models like DistilRoBERTa (Sanh et al., 2019) against the LLM like GPT (Brown et al., 2020) Additionally, the effectiveness of binary classifiers for individual types of TD (such as architecture and testing) was compared against a multiclass approach where a single model attempted to classify all 13 different types of TD simultaneously.

Through this comprehensive methodology, we aimed to identify the most effective models and techniques for TD classification and contribute to a broader understanding of how Transformer-based models can be tailored and applied in software development and maintenance. This would provide actionable insights to help manage TD more effectively, leading to better-maintained and less costly software projects.

*RQ1: Evaluating transformer-based models for TD classification.* To assess the effectiveness of transformer-based models, particularly DistilRoBERTa, in classifying TD:

- **Training:** A binary classification model was trained to distinguish between TD and non-TD issues.
- **OOD Testing:** The model was tested on out-of-distribution (OOD) datasets, including Visma and VSCode projects, to evaluate its generalization capabilities.

*RQ1.1: Impact of project-specific fine-tuning.* This subquestion examines whether fine-tuning on project-specific data improves model performance:

- **Fine-Tuning:** DistilRoBERTa was fine-tuned using 30% of project-specific data from datasets such as Jira and VSCode.
- **Temporal Validation:** A temporal split was applied to test the model's ability to predict TD on future data (2024 GitHub issues).

*RQ2: Comparing GPT models with DistilRoBERTa.* The performance of GPT-based models (e.g., GPT-3.5 Turbo, GPT-4o) was compared to DistilRoBERTa on binary TD classification tasks:

- **Model Selection:** Both fine-tuned DistilRoBERTa and GPT models were evaluated on the same test and OOD datasets.
- **Cost Analysis:** The financial and computational costs of fine-tuning GPT models versus DistilRoBERTa were analyzed.

*RQ2.1: Effectiveness of task-specific fine-tuning for LLMs.* To determine whether task-specific fine-tuning of GPT models outperforms DistilRoBERTa:

- **Training:** GPT4o, GPT-3.5 Turbo and GPT-DaVinci-002 were fine-tuned for TD classification and compared against fine-tuned DistilRoBERTa.
- **Evaluation:** All models were tested on the same benchmark dataset (GH 2024). Metrics such as precision, recall, and MCC were analyzed.

*RQ3: Binary classifiers vs. Multi-class models for TD types.* This question investigates whether binary classifiers outperform multi-class models in detecting 13 TD types:

- **Binary Classifiers:** Individual models were trained for each TD type (e.g., "test debt" vs. "not test debt"). The binary classifiers were integrated into an ensemble, named **BEACon-TD**.
- **Multi-Class Model:** A single multi-class classifier was trained to predict all 13 TD types simultaneously.
- **Observation:** The ensemble of binary classifiers showed consistently higher precision and recall, particularly for rare TD types.

*RQ3.1: Model performance across specific TD types.* The performance of BEACon-TD was compared to GPT-4o across specific TD types:

- **Category-Wise Analysis:** Key metrics (e.g., F1-score, MCC) were computed for each TD type (e.g., "architecture debt", "test debt").
- **Key Findings:** BEACon-TD demonstrated superior performance in most categories except for "people debt", where GPT-4o excelled.

### 3.5. Statistical analysis for model comparison

To rigorously validate the superiority of our model over different GPT's model , we employed a suite of statistical tests, each chosen for its specific strengths in comparing classification models. McNemar's test was selected as our primary test due to its effectiveness in comparing the performance of two models on the same dataset, particularly for **paired nominal data in binary classification tasks**. This test focuses on the disagreements between the two models, providing insight into whether one model significantly outperforms the other (Sundjaja et al., 2023; Dietterich, 1998).

To complement McNemar's test and provide a more comprehensive analysis, we conducted a bootstrap comparison of F1 scores and used 95% Confidence Interval (CI). This non-parametric method was chosen for its robustness in estimating confidence intervals without assuming a specific distribution of the data. The bootstrap analysis allows us to assess the stability and reliability of the performance difference between the two models across multiple resamples of our dataset (Demšar, 2006).

Additionally, we performed a t-test on the F1 scores to compare the mean performance of the two models. While the t-test assumes normality, it provides a well-understood measure of statistical significance that many researchers are familiar with (Student, 1908).

Lastly, we calculated Cliff's Delta for the F1 scores to quantify the effect size of the performance difference. Cliff's Delta was chosen because it is suitable for ordinal data and is less sensitive to outliers compared to other effect size measures (Vargha and Delaney, 2000).

Below, we outline the chosen statistical methods for each RQ and explain how these analyses contribute to answering the respective research questions.

*RQ2: Comparing GPT and DistilRoBERTa for TD classification.* For the comparison between GPT models (e.g., GPT-4o) and fine-tuned DistilRoBERTa models, we used McNemar's test and bootstrap analysis on key metrics such as F1 scores. Cliff's Delta was computed to measure effect size, helping quantify the magnitude of differences in performance. These tests ensured robust conclusions about whether smaller, fine-tuned models could outperform larger, general-purpose models for specific tasks like TD classification.

*RQ2.1: Effectiveness of task-specific fine-tuning for LLMs.* We tested the impact of task-specific fine-tuning on GPT4o, GPT-3.5, GPT Davinci02 and DistilRoBERTa models by comparing their post-fine-tuning performance using paired t-tests and Cliff's Delta. The F1 score confidence intervals were again derived via bootstrap analysis. This approach quantified the relative gains achieved through task-specific adaptation, emphasizing the resource-efficiency trade-offs for each model type.

*RQ3.1: Performance of GPT4o vs. DistilRoBERTa on TD types.* The performance differences between GPT4o and fine-tuned DistilRoBERTa models for specific TD types were analyzed using category-wise McNemar's tests and bootstrap-derived confidence intervals for F1 scores. Individual Cliff's Delta values captured the practical significance of observed differences. These statistical results demonstrated the comparative strengths of each model relative to specific TD categories, such as architectural debt or defect debt.

## 4. Results

Our approach, which we call **BEACon-TD** (Binary Ensemble for Automated Classification of Technical Debt), demonstrated strong performance in identifying and classifying different types of technical debt. BEACon-TD leverages an ensemble of binary classifiers, each specialized in detecting a specific type of technical debt. It offers a simple yet powerful approach to technical debt detection and classification. At its core, BEACon-TD utilizes binary decisions, making it highly adaptable to various project needs. Teams can use the TD classifier alone for quick debt identification, or combine it with specialized classifiers for more nuanced categorization of one of 13 types of TD (Alves et al., 2014).

This flexibility allows practitioners to tailor BEACon-TD to their specific requirements. For instance, they might focus solely on detecting TD, or drill down into particular types like architectural or test debt. The binary nature of BEACon-TD — where an issue must be flagged as both TD and a specific type to be classified — helps minimize false positives while providing detailed insights. This straightforward approach makes BEACon-TD easy to implement, understand, and integrate into existing workflows, offering teams a valuable tool for managing technical debt effectively.

**Table 2**
TD performance metric on TestSet and OOD dataset.

| Metric | TestSet | OOD(va.gov-team) | OOD(VSCode) |
|---|---|---|---|
| Precision | 0.911 | 0.911 | 0.747 |
| Recall | 0.874 | 0.927 | 0.648 |
| Accuracy | 0.894 | 0.918 | 0.718 |
| MCC | 0.789 | 0.837 | 0.439 |
| F1 Score | 0.892 | 0.919 | 0.694 |
| AUC | 0.949 | 0.971 | 0.767 |

**Table 3**
Comparison of before and after fine-tuning (FT) TD model.

| Model | Precision | Recall | Acc | F1 | MCC | AUC |
|---|---|---|---|---|---|---|
| GH 2024 | 0.761 | 0.873 | 0.800 | 0.813 | 0.606 | 0.800 |
| 30% FT | 0.840 | 0.886 | 0.855 | 0.862 | 0.710 | 0.908 |
| JIRA | 0.663 | 0.693 | 0.702 | 0.678 | 0.401 | 0.701 |
| 30% FT | 0.671 | 0.815 | 0.734 | 0.736 | 0.483 | 0.792 |
| VSCode | 0.747 | 0.648 | 0.718 | 0.694 | 0.439 | 0.767 |
| 30% FT | 0.793 | 0.740 | 0.771 | 0.765 | 0.543 | 0.863 |

### *4.1. RQ1: How effective are transformer-based models in classifying TD issues?*

This research question aims to explore the effectiveness of the transformer-based model, particularly DistilRoBERTa in our case, in classifying TD issues, leveraging their ability to capture complex patterns and contextual information in text data. Understanding the effectiveness of these models can guide practitioners in selecting appropriate tools for automated TD management. This investigation will provide empirical evidence on the performance of transformer models in this specific domain, potentially leading to improved accuracy and efficiency in issues classification.

The results in Table 2 demonstrate BEACon-TD (DistilRoBERTa) binary classifier model's performance across different datasets. Notably, the model exhibits consistently high performance on both the testset and the out-of-distribution (OOD) va.gov-team dataset (Department of Veterans Affairs, 2024), with precision, recall, and F1 scores all exceeding 0.90. An Out-Of-Distribution (OOD) dataset is used in machine learning to evaluate models' robustness and generalization ability. OOD datasets contain samples that differ significantly from the data on which the model was trained. This helps to test how well the model can handle new, unseen data that does not fit the distribution of the training set. However, a performance drop is observed on the OOD VSCode dataset (Microsoft, 2024), particularly in the MCC metric, which indicates a decline in the model's ability to correctly classify positive instances. This highlights the challenge of domain shift, where the model's effectiveness diminishes when applied to data that diverges significantly from its training distribution.

These findings underscore the importance of considering domain specificity when deploying machine learning models and emphasize the need for further research into techniques like finetuning or transfer learning to enhance model robustness across diverse datasets.

### *4.2. RQ1.1:Does fine-tuning TD models on project-specific data improve their performance?*

Fine-tuning models on domain-specific or project-specific datasets is a common practice to enhance performance. This research question seeks to determine whether fine-tuning improves the performance of BEACon-TD (DistilRoBERTa) model. The motivation is to assess the gains in classification accuracy and robustness when models are fine-tuned on project-specific or domain-specific TD data. The results will inform how fine-tuning can be beneficial, providing actionable insights for researchers and practitioners looking to optimize their TD classification.

For this study, we extract TD issues from the Jira Public dataset (Montgomery et al., 2022), which is an out-of-distribution (OOD) dataset, as the model was initially trained on GitHub issues. We investigated the generalizability of this OOD Jira dataset to see how well the model can adapt to other project management tools before and after finetuning our TD model.

In addition, we also analyzed the TD model's performance before and after fine-tuning with 30% of VScode-specific TD data issues. The VScode project contains many TD issues and was removed from the initial TD dataset before training, ensuring the model has never encountered VScode-specific instances during training.

We extended our investigation to examine the impact of fine-tuning on the model's temporal generalization capabilities. This aspect is crucial for understanding how well our TD classification models can adapt to and predict future TD issues.

To achieve this, we designed a temporal split in our dataset. We removed all TD-tagged data from GitHub issues dated 2024 (GH 2024) as shown in Table 3 and used this as our test set. The model was then trained exclusively on historical TD and non-TD issues from before 2024. This approach simulates a real-world scenario where models are trained on past data and must predict future occurrences of TD.

By testing the model on the GH 2024 dataset, which it had never encountered during training, we aimed to evaluate its ability to generalize to future temporal data. This experiment allows us to assess how effectively our fine-tuning methods enable the model to capture underlying patterns of TD that persist over time, rather than merely memorizing specific instances or timebound characteristics.

For this RQ, we have assessed improvements in classification accuracy, precision, recall, F1-score, MCC, and AUC ROC . The results, summarized in Table 3, indicate significant performance gains through fine-tuning our BEACon-TD (DistilRoBERTa) model.

Our analysis revealed that fine-tuned models using 30% of project-specific data consistently outperformed their non-fine-tuned counterparts across all datasets. This improvement was evident in various key performance metrics.

For instance, when evaluating on the 'GH 2024' dataset split, which represents future data the model had not seen during training, the fine-tuned model demonstrated substantial improvements. Precision increased from 0.761 to 0.840, indicating a significant reduction in false positives. Recall also improved from 0.873 to 0.886, suggesting better identification of true TD issues. Overall accuracy saw a marked increase from 0.800 to 0.855, reflecting enhanced overall classification performance.

The improvements were even more pronounced for the 'JIRA' (Montgomery et al., 2022) and 'VSCode' (Microsoft, 2024) datasets. In the case of 'JIRA', recall saw a remarkable increase from 0.693 to 0.815, while for 'VSCode', it jumped from 0.648 to 0.740. These significant improvements in recall are particularly important for our TD classification task, as they indicate a better ability to identify true TD instances across diverse project contexts. The consistent enhancement in performance across different datasets underscores the effectiveness of our fine-tuning approach in improving the model's generalization capabilities and its ability to adapt to various software development environments.

Skryseth et al. conducted an empirical evaluation to determine the optimal percentage of project-specific data for fine-tuning TD classification models. Their study examined a range of percentages from 15% to 50% and found that 30% of project-specific data consistently yielded good results across various scenarios (Skryseth et al., 2023). Following their findings, we chose to use 30% of project-specific data for fine-tuning our models. This percentage strikes a balance between leveraging project-specific information and maintaining model generalizability. It provides sufficient data to capture project-specific TD patterns without overfitting to individual project idiosyncrasies (Skryseth et al., 2023). However, it is important to note that practitioners can adjust this percentage based on their specific circumstances and

**Table 4**
Comparison of GPT4o and BEACon-TD DistilRoBERTa model performance metrics.

| Metric | GPT4o | BEACon-TD model | Improvement |
|---|---|---|---|
| Precision | 0.634 | 0.761 | +20.03% |
| Recall | 0.630 | 0.873 | +38.57% |
| Accuracy | 0.633 | 0.800 | +26.38% |
| F1-Score | 0.632 | 0.813 | +28.64% |
| MCC | 0.267 | 0.606 | +126.97% |
| AUC | 0.633 | 0.800 | +26.38% |

data availability. The 30% guideline serves as a well-supported starting point, but the optimal percentage may vary depending on factors such as project size, TD prevalence, and the diversity of TD types within a project. These findings underscore the effectiveness of fine-tuning in enhancing model robustness and accuracy, providing valuable insights for researchers and practitioners aiming to optimize TD classification frameworks.

### *4.3. RQ2: How does the performance of LLM-like the GPT model, compare to TD DistilRoBERTa (BEACon-TD) models in TD classification?*

Large Language Models (LLMs) like GPT and smaller, efficient models like DistilRoBERTa represent different trade-offs in NLP tasks. This research question aims to compare the performance of these two models in the context of TD classification. The motivation is to understand the trade-offs between the computational efficiency of DistilRoBERTa and the expansive capabilities of GPT. The comparison will help identify the optimal model for practical applications, balancing accuracy, computational cost, and resource requirements. The result will clearly explain which model is more suitable for TD classification tasks.

Our study compared the performance of our fine-tuned TD DistilRoBERTa model against the out-of-the-box GPT4o ("o" for "omni") model on the same test set, revealing significant improvements across all metrics as shown in Table 4.

DistilRoBERTa outperformed GPT4o in precision (0.761 vs. 0.634, a 20.03% improvement), recall (0.873 vs. 0.630, a 38.57% increase), and accuracy (0.800 vs. 0.633, a 26.38% gain). The F1-Score for DistilRoBERTa was 0.813, 28.64% higher than GPT4o's 0.632. Notably, the MCC showed the most substantial difference, with DistilRoBERTa scoring 0.606 compared to GPT4o's 0.267, a 126.97% increase. The AUC for DistilRoBERTa (0.800) also surpassed GPT4o (0.633) by 26.38%.

These results demonstrate the superior performance of our fine-tuned DistilRoBERTa model across all key metrics. This comparison underscores the effectiveness of fine-tuning in enhancing model performance and highlights the potential of the smaller, more efficient DistilRoBERTa model in achieving high accuracy in TD classification tasks.

***Statistical testing for BEACon-TD vs. GPT 4o:***. The results from Table 5 collectively provide strong evidence for the superiority of BEACon-TD DistilRoBERTa model. McNemar's test yielded a statistic of 494.0 with a $p$-value of 7.13e−77, decisively rejecting the null hypothesis of no difference between the models. The bootstrap analysis of F1 scores revealed non-overlapping 95% confidence intervals for the two models ([0.8032, 0.8255] for BEACon-TD model vs. [0.6197, 0.6522] for GPT4o).

The t-test's extremely low $p$-value (4.70e−110) further confirms the statistical significance of this performance gap. Finally, the maximum possible Cliff's Delta of 1.0 underscores the consistent and substantial nature of our model's performance advantage.

This comprehensive statistical analysis in Table 5 provides robust evidence that BEACon-TD DistilRoBERTa model significantly outperforms GPT4o across various performance metrics in this binary classification task. The consistency of results across different statistical methods, each chosen for its specific strengths, reinforces the validity of our findings and the substantial improvement offered by our approach.

**Table 5**
Statistical test results: BEACon-TD DistilRoBERTa vs. GPT-4o.

| Test | Results |
|---|---|
| McNemar's test | Statistic: 494.0<br>p-value: 7.13e−77 |
| F1 Score CI (95%) | BEACon-TD: [0.8032, 0.8255]<br>GPT-4o: [0.6197, 0.6522] |
| T-test | p-value: 4.70e−110 |
| Cliff's Delta | 1.0 (Maximum possible) |

**Table 6**
Comparing the GPTs with ours BEACon-TD (DistilRoBERTa) FT model.

| Metric | GPT-3.5 | Davinci | GPT4o | Ours | Our VS. GPT4o (%) |
|---|---|---|---|---|---|
| Precision (%) | 80.6 | 62.1 | 80.68 | 91.1 | 12.92 |
| Recall (%) | 89.9 | 98.3 | 91.03 | 87.4 | −3.99 |
| Accuracy (%) | 84.1 | 69.2 | 84.62 | 89.4 | 5.65 |
| F1-Score (%) | 85.0 | 76.1 | 85.55 | 89.2 | 4.27 |
| MCC (%) | 68.7 | 47.2 | 69.82 | 78.9 | 13.00 |
| AUC (%) | 84.1 | 69.2 | 84.62 | 94.9 | 12.15 |

### *4.4. RQ2.1: Is task specific fine-tuning of LLM like GPT for TD more effective than finetuned DistilRoBERTa TD classification?*

In this RQ, we compares the performance of task-specific fine-tuned GPT models (GPT4o (2024-08-06), GPT-3.5 Turbo and GPT-DaVinci-002) against our task-specific fine-tuned BEACon-TD DistilRoBERTa model for Technical Debt classification on the same Train and test split. We distinguish between two types of fine-tuning:

1. Task-specific fine-tuning: Adapting pre-trained models to the TD classification task.
2. Project-specific fine-tuning: Further adapting models to specific project data.

Our focus is on task-specific fine-tuning, examining whether the extensive pre-training of larger GPT models yields superior performance compared to the smaller, more efficient DistilRoBERTa (Zhuang et al., 2021; Sanh et al., 2019) when both are fine-tuned for TD classification.

We consider the trade-offs between computational resources and model efficiency. GPT models, while powerful, are closed-source and costly for fine-tuning and inference. In contrast, DistilRoBERTa (87 million parameters) can be fine-tuned on smaller GPUs or free resources like Google Colab, and converted to ONNX format for CPU inference (Onnx, 2024).

This investigation aims to guide practitioners in model selection and resource allocation for TD classification tasks, balancing performance gains against computational costs and accessibility.

We compared the performance of task-specific fine-tuned GPT4o (2024-08-06), GPT-3.5 turbo, GPT-DaVinci-002, and BEACon-TD DistilRoBERTa model for Technical Debt classification. To ensure a fair and challenging evaluation, we used a test dataset consisting of GitHub issues from 2024 (GH 2024). This dataset included both TD and non-TD issues that originated after 2024. By using this dataset, we aimed to assess each model's ability to generalize to new, unseen data and to identify TD patterns that persist over time. All models were evaluated on this same GH 2024 dataset, providing a consistent benchmark for comparison. This approach allowed us to test the models' robustness and their capability to adapt to evolving software development practices and TD manifestations. The results revealed that BEACon-TD DistilRoBERTa model generally outperformed the larger GPT models across most metrics as shown in Table 6 .

BEACon-TD demonstrated superior performance in precision, accuracy, F1-Score, and MCC. It achieved the highest precision at 91.1% and highest F1-Score of 89.2%, significantly surpassing both GPT models. In terms of overall accuracy and F1-Score, our model also showed

**Table 7**
Statistical test results for model comparisons.

| Test | BEACon-TD vs. GPT4o | GPT-3.5 vs. Davinci-02 |
|---|---|---|
| McNemar's Test | Statistic: 246.0<br>p-value: 2.50e−11 | Statistic: 233.0<br>p-value: 1.85e−103 |
| F1 Score CI (95%) | BEACon-TD: [0.8757, 0.8979]<br>GPT4o: [0.8437, 0.8665] | GPT-3.5: [0.8406, 0.8593]<br>Davinci-02: [0.7491, 0.7722] |
| T-test | T-statistic: 26.75<br>p-value: 7.63e−47 | T-statistic: 73.08<br>p-value: 2.75e−87 |
| Cliff's Delta | 1.0 (Large effect size) | 1.0 (Large effect size) |

considerable performance improvements, indicating a well-balanced and reliable performance. The MCC, which provides a comprehensive measure of classification quality, was notably higher for DistilRoBERTa, further confirming its robust performance.

While GPT-DaVinci-002 showed the highest recall, BEACon-TD performance in this metric was competitive and close to that of GPT4o and GPT-3.5 turbo. These results suggest that despite the computational advantages often associated with larger models, a smaller, efficiently fine-tuned model like DistilRoBERTa can achieve superior performance in specialized tasks such as TD classification. This finding emphasizes the potential cost–benefit advantage of using more resource-efficient models without significant performance loss in practical applications.

***Statistical testing for comparing GPT4o vs. GPT 3.5 vs. GPT Davinci 02 vs. BEACon-TD:***. Our study conducted a comprehensive comparison of three fine-tuned models: BEACon-TD, GPT4o, GPT-3.5, and Davinci, using a suite of statistical tests to evaluate their relative performance on the same dataset as shown in Table 7. The results consistently demonstrated a clear performance hierarchy, with BEACon-TD DistilRoBERTa outperforming GPT4o, GPT-3.5 and Davinci-02, and GPT-3.5 in turn surpassing Davinci-02 model. In the comparison between BEACon-TD and GPT4o, McNemar's test revealed a significant difference (statistic: 246.0, p-value: 2.50e−11), while bootstrap analysis of F1 scores showed non-overlapping confidence intervals (DistilRoBERTa : [0.8757, 0.8979], GPT-3.5: [0.8437, 0.8665]).

The t-test further confirmed this significant difference (T-statistic: 26.75, p-value: 7.63e−47), and Cliff's Delta indicated a large effect size (1.0). Similar patterns were observed in the comparison between GPT-3.5 and Davinci, with even more pronounced differences (McNemar's statistic: 233.0, p-value: 1.85e−103; F1 score CIs (Confidence Interval 95%) : GPT-3.5 [0.8406, 0.8593], Davinci [0.7491, 0.7722]). The consistency of these results across multiple statistical methods strengthens the reliability of our findings. Notably, the large effect sizes (Cliff's Delta = 1.0) in both comparisons indicate that the performance differences are not only statistically significant but also substantial and consistent across the dataset.

These findings in Table 7 highlight the effectiveness of BEACon-TD fine-tuning approach with DistilRoBERTa, which has resulted in a model that outperforms even larger, more resource-intensive models like GPT4o, GPT-3.5 and Davinci on this specific task. This demonstrates the potential for smaller, more efficient models to achieve state-of-the-art performance when properly fine-tuned for specialized tasks, potentially offering significant advantages in terms of computational efficiency and deployment flexibility.

### *4.5. RQ3: How effective are expert ensemble of binary classifiers in classifying different types of issues compared to multi-class model?*

#### *4.5.1. Evaluation binary classifier*

This research question investigates the effectiveness of classification models in distinguishing among 13 different types of software issues, as identified by Alves et al. (2014). These issue types are not explicitly labeled as TD but represent categories often associated with TD. For the TD identification, we employ a separate expert binary TD model.

Our primary goal is to improve the granularity of issue identification, which could lead to more targeted remediation strategies, whether these issues represent TD or not. We compare two approaches:

**Table 8**
Test set and OOD performance metrics for various categories by binary classifiers.

| Category | Test Set | | | OOD | | |
|---|---|---|---|---|---|---|
| | MCC | F1 | Recall | MCC | F1 | Recall |
| Architecture | 0.616 | 0.809 | 0.817 | 0.803 | 0.903 | 0.992 |
| Automation | 0.680 | 0.836 | 0.819 | −0.124 | 0.096 | 0.057 |
| Build | 0.760 | 0.880 | 0.878 | 0.836 | 0.920 | 0.957 |
| Code | 0.592 | 0.793 | 0.780 | −0.027 | 0.246 | 0.166 |
| Defect | 0.865 | 0.932 | 0.925 | 0.860 | 0.930 | 0.925 |
| Design | 0.692 | 0.847 | 0.854 | 0.780 | 0.893 | 0.939 |
| Documentation | 0.773 | 0.887 | 0.889 | 0.719 | 0.857 | 0.846 |
| Infrastructure | 0.642 | 0.820 | 0.815 | 0.743 | 0.876 | 0.913 |
| People | 0.769 | 0.887 | 0.907 | 0.887 | 0.944 | 1.000 |
| Process | 0.605 | 0.800 | 0.792 | −0.010 | 0.250 | 0.168 |
| Requirement | 0.678 | 0.843 | 0.865 | 0.774 | 0.891 | 0.951 |
| Service | 0.813 | 0.906 | 0.897 | 0.892 | 0.947 | 0.974 |
| Test | 0.731 | 0.864 | 0.851 | 0.862 | 0.932 | 0.977 |

1. A multiclass classification model that attempts to categorize issues into all 13 types simultaneously.
2. Expert binary classifiers, each trained to identify a specific issue type (e.g., "architectural" vs. "not architectural").

The dataset for this study was curated by mining GitHub issues labeled with keywords corresponding to various tissue types, such as Architecture, Automation, Build, and Code. We divided this data into training, test, and separate out-of-distribution (OOD) projects not included in the training phase.

We assess the performance of both multiclass and binary models on the test set and the OOD dataset. This evaluation aims to determine which approach — multiclass or multiple expert binary classifiers — is more effective in distinguishing between different issue types.

Table 8 presents performance metrics for various issue categories on both the test set and out-of-distribution (OOD) data, focusing on MCC, F1 score, and Recall using DistilRoBERTa binary classifier.

On the test set, most categories show strong performance with MCC values typically above 0.6 and F1 scores ranging from 0.8 to 0.9. Categories like "Defect", "Service", and "Build" perform exceptionally well, with MCC values around or above 0.8.

However, OOD data reveals significant performance variations. Categories such as "Architecture", "Build", "Defect", "Service", and "People" maintain high MCC values, F1 scores, and Recall, indicating good generalization. In contrast, "Automation", "Code", and "Process" show sharp declines in all metrics, with negative MCC values suggesting poor performance on OOD data.

These results highlight the variability in model generalization across different issue categories. While some models demonstrate robust performance beyond their training data, others struggle significantly with OOD samples. This disparity underscores the need for targeted improvements in specific categories to enhance model robustness and applicability in diverse, real-world scenarios.

#### *4.5.2. Evaluation and comparison of multi-class classifier*

We evaluate the effectiveness of multi-class classifiers in distinguishing between 13 different types of software issues often associated

**Table 9**
Performance metrics for **Multi-Class (MC)** and binary models on various categories.

| Metric | MC test | MC OOD | Binary test | | Binary OOD | |
|---|---|---|---|---|---|---|
| | | | Mean | SD | Mean | SD |
| Precision | 0.666 | 0.588 | 0.855 | 0.043 | 0.771 | 0.205 |
| Recall | 0.648 | 0.593 | 0.853 | 0.046 | 0.759 | 0.361 |
| Accuracy | 0.761 | 0.627 | 0.854 | 0.042 | 0.808 | 0.187 |
| MCC | 0.710 | 0.561 | 0.709 | 0.084 | 0.615 | 0.386 |
| F1 xcore | 0.655 | 0.554 | 0.854 | 0.043 | 0.745 | 0.315 |
| AUC | 0.929 | 0.859 | 0.923 | 0.030 | 0.865 | 0.202 |

with TD, such as Architecture, Code, and Build. It is important to note that the multi-class model is trained to identify issue categories, not the presence of TD itself. We compare this multi-class approach with the binary classifiers. The motivation is to understand the potential of multi-class models in providing comprehensive and accurate issue-type identification.

This comparison aims to identify the relative strengths and weaknesses of each approach. Multi-class (MC) classification offers a holistic view but may face challenges in complexity and computational costs. Binary classifiers, while simpler and potentially more interpretable, might be less comprehensive when considering all issue types simultaneously. The findings will contribute to developing more sophisticated classification systems and improving software quality assessment and maintainability strategies.

Table 9 shows the multi-class (MC) DistilRoBERTa classifier's performance, with test set metrics of Precision (0.666), Recall (0.648), Accuracy (0.761), MCC (0.710), F1 Score (0.655), and AUC (0.929). Out-of-distribution (OOD) results are slightly lower, indicating some challenges in generalization. Table 9 also summarizes the performance of binary DistilRoBERTa classifiers for 13 TD types from results in Table 8 with mean and Standard Deviation (SD). On the test set, mean metrics include Precision (0.855), Recall (0.853), Accuracy (0.854), MCC (0.709), F1 Score (0.854), and AUC (0.923). OOD performance shows more variability but remains strong.

Comparing the two approaches:

- Binary classifiers demonstrate higher precision and recall in both test and OOD datasets.
- Binary models achieve higher accuracy, suggesting more reliable distinction between TD types.
- Binary classifiers outperform the multi-class model in MCC and F1 score.

Overall, binary classifiers exhibit superior performance metrics. This suggests that combining an ensemble of binary classifier categorization with expert binary TD models could leverage the strengths of both approaches, potentially enhancing overall TD identification and management.

### *4.6. RQ 3.1: How does the performance of LLM-like the GPT model compare to the DistilRoBERTa (BEACon-TD) model on different issue types ?*

Table 10 compares performance metrics between BEACon-TD fine-tuned DistilRoBERTa model and GPT-4o ("**o**" for "omni") across various issues categories.

We compare the performance of our BEACon-TD fine-tuned DistilRoBERTa model with GPT-4o across various issue categories commonly associated with software development and potential Technical Debt types. Both models were evaluated on a 2024 dataset to ensure fair comparison and avoid data leakage, as neither model was trained on this data. BEACon-TD DistilRoBERTa model demonstrated superior performance in most technical categories. For example, in 'Defect' detection, it achieved an F1-Score of 0.923, significantly outperforming GPT-4's 0.760. This trend was consistent across several

**Table 10**
Comparison of performance metrics between BEACon-TD and GPT-4o.

| Category | BEACon-TD | | | GPT-4o | | |
|---|---|---|---|---|---|---|
| | F1-Score | MCC | Recall | F1-Score | MCC | Recall |
| Arch | 0.803 | 0.622 | 0.773 | 0.758 | 0.543 | 0.720 |
| Auto | 0.766 | 0.582 | 0.700 | 0.759 | 0.551 | 0.713 |
| Build | 0.881 | 0.767 | 0.867 | 0.875 | 0.753 | 0.867 |
| Code | 0.703 | 0.459 | 0.647 | 0.675 | 0.165 | 0.920 |
| Defect | 0.921 | 0.848 | 0.893 | 0.760 | 0.466 | 0.980 |
| Design | 0.904 | 0.814 | 0.880 | 0.809 | 0.645 | 0.760 |
| Doc | 0.886 | 0.768 | 0.907 | 0.868 | 0.749 | 0.833 |
| Infra | 0.769 | 0.562 | 0.733 | 0.719 | 0.370 | 0.827 |
| People | 0.944 | 0.889 | 0.944 | 0.971 | 0.946 | 0.944 |
| Process | 0.751 | 0.566 | 0.673 | 0.714 | 0.412 | 0.738 |
| Require | 0.836 | 0.681 | 0.813 | 0.785 | 0.582 | 0.787 |
| Service | 0.861 | 0.746 | 0.807 | 0.800 | 0.572 | 0.880 |
| Test | 0.897 | 0.793 | 0.900 | 0.794 | 0.618 | 0.747 |

technical categories, indicating our model's strength in areas requiring high technical accuracy and automation. However, GPT-4 excelled in the 'People' category, with an F1-Score of 0.971 compared to our model's 0.944. These results highlight the effectiveness of BEACon-TD fine-tuned DistilRoBERTa model in classifying all TD issue types.

***Statistical testing BEACon-TD vs. GPT-4o:***. Our study compared the performance of a fine-tuned BEACon-TD model against GPT-4o across 13 different categories typically associated with TD and other software engineering issues. We employed multiple statistical tests to ensure a comprehensive analysis: McNemar's Test, Bootstrap Comparison of F1 Scores with 95% CI (Confidence Interval), T-test on F1 Scores, and Cliff's Delta for effect size.

The results in Table 11 demonstrate that BEACon-TD DistilRoBERTa model generally outperforms or matches GPT-4o across most categories. Specifically:

1. DistilRoBERTa significantly outperformed GPT-4o in 5 categories: defect, design, requirement, service, and test.
2. In 4 categories (code, documentation, infrastructure, process), some tests showed significant differences favoring DistilRoBERTa, while others were inconclusive.
3. For 3 categories (architecture, automation, build), the results were mixed or showed no significant difference.
4. Only in the 'people' category did some tests suggest GPT-4o might perform better, though results were inconsistent across tests.

Table 11 below presents the detailed numeric values for each statistical test:

These results suggest that BEACon-TD DistilRoBERTa model is competitive with, and often superior to, GPT-4o across a wide range of software engineering tasks. The consistent large effect sizes in many categories further emphasize the practical significance of these performance differences. This demonstrates the potential of smaller, efficiently fine-tuned models to match or exceed the performance of larger, more general models in specialized domains.

## 5. Evaluation

In our evaluation study, we focused on two main classification tasks:

1. Categorizing software issues into 13 types commonly associated with TD, such as architecture, build, and test.
2. Classifying and identifying whether an issue represents TD or Not using an expert TD Binary model.

We created a ground truth dataset consisting of issues tagged with both a specific category label and a TD label. For example, an issue might be labeled as both "Architecture" and "Technical Debt". To ensure a fair

**Table 11**
Detailed statistical test results.

| Category | McNemar's Stat (p-value) | DistilRoBERTa F1 CI | GPT-4o F1 CI | T-stat (p-value) | Cliff's Delta |
|---|---|---|---|---|---|
| Architecture | 28.0 (0.182) | [0.750, 0.840] | [0.697, 0.806] | 7.590 (1.88e−11) | 0.736 |
| Automation | 32.0 (0.716) | [0.695, 0.826] | [0.711, 0.820] | 1.261 (0.210) | 0.175 |
| Build | 15.0 (0.860) | [0.829, 0.912] | [0.841, 0.905] | 2.115 (0.037) | 0.258 |
| Code | 52.0 (5.12e−5) | [0.651, 0.758] | [0.616, 0.725] | 5.264 (8.31e−7) | 0.525 |
| Defect | 14.0 (2.46e−13) | [0.889, 0.946] | [0.709, 0.799] | 41.642 (4.00e−64) | 1.000 |
| Design | 17.0 (0.001) | [0.877, 0.935] | [0.756, 0.868] | 19.331 (3.28e−35) | 0.998 |
| Documentation | 18.0 (0.749) | [0.850, 0.913] | [0.827, 0.904] | 4.491 (1.94e−5) | 0.489 |
| Infrastructure | 31.0 (0.002) | [0.708, 0.809] | [0.648, 0.774] | 8.229 (8.22e−13) | 0.762 |
| People | 1.0 (1.000) | [0.866, 1.000] | [0.890, 1.000] | −3.483 (0.001) | −0.402 |
| Process | 42.0 (0.050) | [0.683, 0.797] | [0.650, 0.767] | 4.725 (7.69e−6) | 0.492 |
| Requirement | 21.0 (0.001) | [0.786, 0.874] | [0.729, 0.806] | 11.918 (8.96e−21) | 0.923 |
| Service | 24.0 (0.002) | [0.817, 0.908] | [0.761, 0.841] | 13.745 (1.36e−24) | 0.936 |
| Test | 12.0 (0.0002) | [0.857, 0.924] | [0.744, 0.831] | 21.610 (4.63e−39) | 1.000 |

evaluation, we excluded these doubly-tagged issues from our training data and used them as a separate test set. We then evaluated three types of models:

1. A Multi-Class (MC) model that categorizes issues into all 13 types simultaneously.
2. 13 separate binary models, each specialized in identifying one specific issue type.
3. A single expert binary model focused solely on identifying TD.

This approach allowed us to compare the effectiveness of different classification strategies and assess how well each model type performs in identifying specific issue categories and TD.

We used the recall metric to evaluate model performance. Recall was chosen because our ground truth dataset only contains positive examples (issues tagged with both a specific TD category and TD). we generally assume that issues not explicitly tagged as technical debt (TD) serve as negative examples, because issues not tagged as TD are typically treated as negative examples in the training and evaluation process. This allows us to create a binary classification problem where models can learn to distinguish between TD and non-TD issues. Recall measures how many of these known positive instances were correctly identified by each model. For instance, if we have 100 issues tagged as both "Architecture" and "TD", and our model correctly identifies 80 of them, the recall would be 0.8 or 80%. The support metric indicates the total number of these doubly-tagged issues for each category, providing context for the recall scores. This approach allowed us to assess how well our models could identify specific issue categories and TD, using a carefully curated set of real-world, developer-labeled issues as our benchmark. Table 12 compares the performance of our DistilRoBERTa multi-class (MC), binary issues categories type, and expert TD only classifiers in identifying various types of technical debt. The results show that binary classifiers generally outperform the multi-class approach in terms of recall across most TD types. Key findings:

- Binary classifiers consistently achieve higher recall than the multi-class (MC) model. For example, in Architecture Debt detection, recall values are 0.160 (MC), 0.755 (binary architecture issues type), and 0.980 (expert TD only).
- The expert TD binary model demonstrates consistently high recall across all categories, highlighting the effectiveness of specialized models.
- Support values vary significantly across categories, influencing the reliability of recall metrics. High-support categories like Test Debt (1950 instances) offer more robust results, while low-support categories like People Debt (1 instance) may yield less reliable metrics.

These results suggest that an ensemble of binary classification, with specialized models for each TD type and expert TD-only model, is more effective than a multi-class approach for technical debt identification.

**Table 12**
OOD test TD types classification of our DistilRoBERTa.

| Category | Recall | | | Support |
|---|---|---|---|---|
| | MC | Binary types | TD only | |
| Architecture Debt | 0.160 | 0.755 | 0.980 | 49 |
| Automation Debt | 0.470 | 0.975 | 0.980 | 40 |
| Build Debt | 0.600 | 0.829 | 0.947 | 246 |
| Code Debt | 0.610 | 0.821 | 0.918 | 469 |
| Defect Debt | 0.740 | 0.800 | 0.980 | 50 |
| Design Debt | 0.650 | 0.800 | 0.975 | 670 |
| Documentation Debt | 0.790 | 0.803 | 0.967 | 528 |
| Infrastructure Debt | 0.300 | 0.774 | 0.954 | 195 |
| People Debt | 0.000 | 0.856 | 0.930 | 1 |
| Process Debt | 0.210 | 0.727 | 0.939 | 66 |
| Requirement Debt | 0.230 | 0.736 | 0.970 | 61 |
| Service Debt | 0.670 | 0.790 | 0.984 | 433 |
| Test Debt | 0.670 | 0.825 | 0.972 | 1950 |

**Table 13**
Classification metrics for technical debt before and after 30% fine-tuning with Visma dataset.

| Metric | Before fine-tuning | After 30% fine-tuning |
|---|---|---|
| Accuracy | 0.568 | 0.737 |
| Precision | 0.451 | 0.608 |
| Recall | 0.791 | 0.801 |
| F1-Score | 0.574 | 0.691 |
| MCC | 0.664 | 0.707 |
| AUC | 0.615 | 0.750 |

### *5.1. Evaluation of industry projects*

We evaluated the BEACon-TD DistilRoBERTa TD binary classification model on a dataset from Visma, a multinational software company comprising over 180 entrepreneurial software companies. Visma's unique structure, with high autonomy among its companies and diverse team practices, makes it an ideal candidate for testing the model's effectiveness in identifying and classifying TD across varied project contexts. This dataset from Visma consisted of approximately 1100 Jira issues from 6 different projects across 3 countries, with about 400 issues labeled as TD. This diversity in data sources ensures a robust test of the model's generalization capabilities. Table 13 presents our DistilRoBERTa model's performance before and after fine-tuning with 30% of the Visma dataset:

The results show significant improvements across all metrics after fine-tuning. Notably, accuracy increased from 56.8% to 73.7%, and precision from 45.1% to 60.8%. The recall, crucial for identifying potential TD issues, was already high at 79.1% and improved slightly to 80.1%. In this context, recall is particularly important. Given the inconsistent TD tagging practices across different teams and projects, the model's ability to identify potential TD issues, even those not explicitly tagged, is vital. A high recall ensures that most TD issues are captured, allowing teams to address them proactively. The improvements in F1-Score (57.4% to 69.1%), MCC (66.4% to 70.7%), and AUC (61.5% to

75.0%) further demonstrate the model's enhanced effectiveness after fine-tuning. These results are promising for Visma's diverse software ecosystem. The model's improved performance in identifying TD issues, even in previously untagged instances, can significantly aid in standardizing TD management practices across Visma's various companies. This approach enhances software maintenance practices and contributes to more sustainable and consistent software development processes within Visma's complex corporate environment.

## 6. Discussion

BEACon-TD models enable automated analysis of TD patterns across large-scale software repositories. These models support both broad studies of TD evolution across thousands of projects and focused analyses within specific organizations. The ability to fine-tune models on project-specific data allows companies to customize TD detection according to their unique requirements and priorities.

Our key technical contributions include: (1) an ensemble of binary classifiers that improves classification accuracy across multiple TD types, building upon Tan et al.'s work, and (2) the first comprehensive comparison between smaller BERT-based models (DistilRoBERTa) and GPT models (GPT4o, GPT 3.5) for TD classification. This advances existing TD detection approaches by providing both broader coverage of TD types and higher accuracy than previous single-architecture studies (Flisar and Podgorelec, 2021).

These advances enable new research directions in studying TD evolution patterns and development practices, while offering practical tools for industry application in software development processes.

### 6.1. Implications for research and industry

***Large-scale TD studies.*** Our BEACon-TD models enable reliable tagging of large numbers of issues, facilitating comprehensive studies on TD patterns and evolution. Researchers could use our models to analyze TD trends across thousands of open-source projects on GitHub, providing insights into common TD types and their prevalence across different programming languages or project domains (Bavota and Russo, 2016).

The effectiveness of our approach across different projects enables comparative studies of TD across diverse software ecosystems. For example, researchers could compare TD patterns in mobile app development versus enterprise software projects, identifying domain-specific TD challenges.

Our high-performance models, with potential for further optimization, open new avenues for TD research. Future work could explore combining our approach with other techniques, such as static code analysis or dynamic runtime monitoring, to create even more robust TD detection systems. This could lead to more nuanced TD classification, potentially identifying subtypes of TD or predicting the long-term impact of specific TD instances.

***Temporal analysis of TD.*** The ability of our models to generalize to future data opens possibilities for studying TD evolution over time. Researchers could track how TD accumulates in long-running projects, potentially developing predictive models for TD growth based on project characteristics and development practices.

This temporal analysis could be extended to investigate the relationship between TD and project outcomes. For instance, researchers could study how early TD accumulation affects project success rates, time-to-market, or long-term maintenance costs. Such insights could inform best practices for TD management throughout the software development lifecycle.

***Implication for software industry.*** Our research offers several practical benefits for the software industry:

***Retroactive TD tagging.*** Our approach allows for retroactively tagging TD in existing projects. This is valuable for scenarios such as (Ernst et al., 2021):

- Postmortem analysis of completed projects to identify areas for improvement in future developments.
- Due diligence during company acquisitions, providing insight into the acquired software's technical health.
- Project handovers, ensuring the new team understands the existing TD landscape.

***Efficient TD management.*** The deployment of our models can streamline TD detection and management in large projects. For instance, a development team could integrate our model into their CI/CD pipeline, automatically flagging potential TD issues during code reviews and prioritizing them based on severity (Skryseth et al., 2023).

***Customizable TD detection.*** The ability to fine-tune models on project-specific data allows companies to tailor TD detection to their contexts. A financial services company, for example, could fine-tune the model to be particularly sensitive to security-related TD, reflecting their industry's specific concerns.

### 6.2. Integration with software economics and project management

Our TD classification approach has broader implications for software economics and project management:

***TD quantification and economic impact.*** Future research could develop methods to quantify the economic impact of identified TD. By estimating the cost of addressing TD versus the long-term savings from reduced maintenance efforts, organizations could make more informed decisions about TD remediation priorities (Lenarduzzi et al., 2021; Tsoukalas et al., 2018; Arvanitou et al., 2022).

This economic perspective could be integrated into project management tools, providing real-time cost–benefit analyses of addressing specific TD instances. Such integration could help project managers and developers make data-driven decisions about when and how to address TD.

***TD in Agile methodologies.*** Investigating how our automated TD classification can be integrated into agile processes could provide insights into balancing short-term delivery with long-term code quality. TD metrics could be incorporated into sprint planning and retrospectives, ensuring TD is consistently addressed alongside new feature development (Santos et al., 2022).

This integration could lead to the development of new agile practices specifically designed to manage TD effectively. For example, "TD sprints" or "refactoring story points" could become standard elements of agile methodologies, ensuring that TD is given appropriate attention in fast-paced development environments.

***TD and software metrics.*** Exploring correlations between our TD classifications and traditional software metrics could provide a more holistic view of software quality. Researchers could investigate how TD correlates with metrics like cyclomatic complexity or coupling, potentially developing more comprehensive software quality models (Tsoukalas et al., 2018; Arvanitou et al., 2022).

This holistic approach could lead to the development of new, TD-aware software metrics that provide a more nuanced understanding of code quality. Such metrics could be invaluable for both researchers studying software evolution and practitioners seeking to improve their development practices.

### 6.3. The importance of recall in TD detection

In TD classification, recall plays a crucial role. High recall ensures that most TD instances are identified, minimizing the risk of overlooking critical issues. Our models achieved a balance between precision and recall, with a slight emphasis on recall. This approach is particularly valuable in large-scale projects where manual review of all potential TD is impractical.

The high recall of our model enables a trustworthy TD detection process. For instance, in a large enterprise software project with thousands of issues, our model might identify 80% of actual TD instances. This high recall allows us to confidently use the model to tag previously unidentified TD. Many of the model's apparent "false positives" are likely to be genuine TD instances that developers initially overlooked or failed to tag. As a result, our model serves a dual purpose: it not only detects known TD but also uncovers hidden TD that might have otherwise gone unnoticed. This capability provides a more comprehensive and accurate view of the project's technical debt landscape.

By leveraging the model's high recall, development teams can systematically review and validate these potential TD instances, effectively using the model as a tool to enhance their TD identification process. This approach can lead to a more thorough and consistent TD management strategy, ensuring that a larger proportion of actual TD is captured and addressed throughout the project lifecycle.

### 6.4. Limitations and future work

Our study, while promising, has limitations. The keyword-based dataset curation might miss implicit TD instances, suggesting future exploration of advanced NLP techniques for comprehensive TD identification.

While we evaluated models on OOD datasets, further testing on diverse real-world projects is needed to confirm robustness and generalizability. Future work should involve live deployments and developer feedback to refine the system (Avgeriou et al., 2023).

Although our ensemble learning approach with binary classifiers showed promising results, there is room for optimization. Future research could explore advanced ensemble techniques like stacking or boosting to enhance predictive performance.

Expanding the dataset to include a wider range of sources and project types would ensure broader model applicability, addressing the current limitations in project diversity representation.

Our current approach focuses on TD identification. Future work could explore methods to assess the impact of identified TD on project outcomes, potentially developing predictive models for the long-term consequences of specific TD types.

## 7. Threats to validity

In our study, we identify threats to validity that could impact the results and interpretations.

A key threat to internal validity is the potential for selection bias in our dataset. Our dataset comprises issues tagged by developers with terms related to technical debt, which may not represent all instances of TD comprehensively. Additionally, the accuracy of our classifiers might be influenced by the quality of the annotations. Any inconsistencies or inaccuracies in tagging could affect the model's performance. Moreover, the use of transformer-based models, which require substantial computational resources, might introduce variance in results due to differences in training conditions and hyperparameter settings.

The generalizability of our findings is a significant concern. Our study uses a dataset curated from GitHub projects, which might not represent all types of software projects or development environments. This limitation could affect the applicability of our results to other domains or projects with different characteristics. Additionally, the models' performance on out-of-distribution (OOD) data needs careful interpretation, as it may not accurately reflect real-world scenarios where data characteristics differ significantly from the training set.

However, it is important to note that we have taken steps to address this concern. We evaluated our models on multiple OOD projects and, notably, on industrial projects from a multinational software company. These evaluations yielded promising results, suggesting a degree of generalizability beyond our initial dataset.

Construct validity threats include the definitions and measurements of technical debt types. While we rely on established ontologies, the subjective nature of tagging and the varying interpretations of what constitutes technical debt could introduce bias. Furthermore, our approach to classifying TD types using binary classifiers assumes clear distinctions between types, which may not always hold true in practice. The overlap between different types of technical debt could affect the accuracy of our classifications and the conclusions drawn from them.

The potential overlap between different types of technical debt could affect the granularity of our classifications to some degree. This overlap might lead to slight inaccuracies in distinguishing between closely related TD types but is unlikely to significantly impact the overall validity of our conclusions.

To address the threats to validity in our study, we implemented several mitigation strategies. For dataset selection bias, we employed comprehensive regex patterns to capture diverse TD representations and created a separate ground truth dataset with explicitly tagged issues. We conducted rigorous data cleaning and preprocessing, including balancing datasets to ensure equal proportions of positive and negative labels, reducing the risk of bias in our models.

To minimize variance in results from transformer-based models, we used consistent training protocols, including 5-fold cross-validation over five epochs, and compared multiple model architectures. To enhance generalizability, we evaluated our models on multiple out-of-distribution (OOD) datasets, including industrial projects from a multinational software company. We created separate OOD datasets for each TD category and evaluated our models on a diverse dataset from Visma.

To address construct validity threats, we based our TD type classifications on the established ontology by Alves et al. (2014). We employed ensemble learning techniques, combining multiple binary classifiers to enhance the robustness of our TD type identification. To handle potential overlaps between TD types, we compared our binary classification approach with a multiclass model and focused on recall as a key metric. We also conducted a detailed analysis of model performance across different TD types to identify and discuss areas where overlaps might occur.

These mitigation strategies collectively enhance the robustness and reliability of our findings, addressing the key threats to validity identified in our study.

## 8. Conclusion

In this study, we demonstrate advancements in identifying various TD types from issue trackers and project management systems. We found that fine-tuning transformer-based models on project specific TD data significantly enhances their performance. Our comparison between GPT and our BEACon-TD DistilRoBERTa models revealed that while large language models like GPT show promise, our fine-tuned DistilRoBERTa model often outperformed them. This highlights the importance of targeted fine-tuning over sheer model size in TD classification tasks

Notably, our expert binary classifiers consistently outperformed multi-class approaches, providing more precise and reliable identification of specific TD types. This improvement is particularly evident in our ability to distinguish between closely related TD categories, a challenge in previous studies.

A key strength of our study is the extensive evaluation in both open-source and industrial settings. Our BEACon-TD models demonstrated robust performance on out-of-distribution data, including a real-world dataset from a multinational software company. This industrial validation underscores the practical applicability of our approach in diverse and evolving software projects.

By releasing our curated dataset and presenting these improved methods, we aim to drive further advancements in TD classification research. Our work not only enhances the accuracy of TD detection but also provides a more nuanced understanding of different TD types, contributing to more effective software maintenance and quality assurance strategies in both academic and industrial contexts.

## CRediT authorship contribution statement

**Karthik Shivashankar:** Writing – review & editing, Writing – original draft, Visualization, Validation, Software, Resources, Methodology, Investigation, Formal analysis, Data curation, Conceptualization. **Mili Orucevic:** Writing – review & editing, Validation, Resources, Project administration, Funding acquisition, Data curation. **Maren Maritsdatter Kruke:** Writing – review & editing, Validation, Resources, Project administration, Investigation, Data curation. **Antonio Martini:** Writing – review & editing, Validation, Supervision, Project administration, Methodology, Funding acquisition, Conceptualization.

## Declaration of Generative AI and AI-assisted technologies in the writing process

During the preparation of this work the authors used Claude from Anthropic in order to rewrite for clarity and improve readability. After using this tool, the authors reviewed and edited the content as needed and takes full responsibility for the content of the published article.

## Declaration of competing interest

The authors declare the following financial interests/personal relationships which may be considered as potential competing interests: Mili Orucevic reports financial support was provided by Visma Software International. Mili Orucevic reports financial support was provided by Research Council of Norway. Maren Maritsdatter Kruke reports financial support was provided by Visma Software International. Maren Maritsdatter Kruke reports financial support was provided by Research Council of Norway. Karthik Shivashankar reports financial support was provided by Research Council of Norway. Antonio Martini reports financial support was provided by Research Council of Norway. If there are other authors, they declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

## Acknowledgment

This project has received funding from the Research Council of Norway's Innovation Project for the Industrial Sector (IPN) Programme under grant agreement for project No. 340991.

## Data availability

Most of the data is shared in Zenodo. The industry project data cannot be shared for confidentiality reasons.

## References

Alves, N.S., Ribeiro, L.F., Caires, V., Mendes, T.S., Spínola, R.O., 2014. Towards an ontology of terms on technical debt. In: 2014 Sixth International Workshop on Managing Technical Debt. pp. 1–7. http://dx.doi.org/10.1109/MTD.2014.9.

Arvanitou, E.-M., Argyriadou, P., Koutsou, G., Ampatzoglou, A., Chatzigeorgiou, A., 2022. Quantifying TD interest: Are we getting closer, or not even that? In: 2022 48th Euromicro Conference on Software Engineering and Advanced Applications. SEAA, IEEE, pp. 274–281.

Avgeriou, P., Ozkaya, I., Chatzigeorgiou, A., Ciolkowski, M., Ernst, N.A., Koontz, R.J., Poort, E., Shull, F., 2023. Technical debt management: The road ahead for successful software delivery. In: 2023 IEEE/ACM International Conference on Software Engineering: Future of Software Engineering. ICSE-FoSE, IEEE, pp. 15–30.

Bavota, G., Russo, B., 2016. A large-scale empirical study on self-admitted technical debt. In: Proceedings of the 13th International Conference on Mining Software Repositories. pp. 315–326.

Besker, T., Ghanbari, H., Martini, A., Bosch, J., 2020. The influence of technical debt on software developer morale. J. Syst. Softw. 167, 110586.

Bilal, M., Almazroi, A.A., 2023. Effectiveness of fine-tuned BERT model in classification of helpful and unhelpful online customer reviews. Electron. Commer. Res. 23 (4), 2737–2757.

Brown, T., Mann, B., Ryder, N., Subbiah, M., Kaplan, J.D., Dhariwal, P., Neelakantan, A., Shyam, P., Sastry, G., Askell, A., et al., 2020. Language models are few-shot learners. Adv. Neural Inf. Process. Syst. 33, 1877–1901.

Chicco, D., Jurman, G., 2020-01-02. The advantages of the matthews correlation coefficient (MCC) over F1 score and accuracy in binary classification evaluation. BMC Genom. 21 (1), 6, URL https://doi.org/10.1186/s12864-019-6413-7.

Cruz, D., Santana, A., Figueiredo, E., 2020. Detecting bad smells with machine learning algorithms: an empirical study. In: 2020 IEEE/ ACM International Conference on Technical Debt. TechDebt, pp. 31–40, URL https://api.semanticscholar.org/CorpusID:221911206.

Demšar, J., 2006. Statistical comparisons of classifiers over multiple data sets. J. Mach. Learn. Res. 7 (1), 1–30, URL http://jmlr.org/papers/v7/demsar06a.html.

Department of Veterans Affairs, 2024. VA.gov-team. https://github.com/department-of-veterans-affairs/va.gov-team. (Accessed 28 June 2024).

Devlin, J., Chang, M.-W., Lee, K., Toutanova, K., 2018. Bert: Pre-training of deep bidirectional transformers for language understanding. arXiv preprint arXiv:1810.04805.

Dietterich, T.G., 1998. Approximate statistical tests for comparing supervised classification learning algorithms. Neural Comput. 10 (7), 1895–1923. http://dx.doi.org/10.1162/089976698300017197.

Ernst, N., Kazman, R., Delange, J., 2021. Technical Debt in Practice: How to Find It and Fix It. MIT Press.

Faus, J.B., Masson, P.L., Pelissier, U., Jibet, N., Bordas, A., Pajot, S., 2023. Design methods for diagnosing and locating entangled technical debt in devops frameworks. Proc. Des. Soc. 3, 1267–1276, URL https://api.semanticscholar.org/CorpusID:259190189.

Flisar, J., Podgorelec, V., 2021. Deep learning for self-admitted technical debt identification. In: 2021 IEEE 45th Annual Computers, Software, and Applications Conference. COMPSAC, IEEE, pp. 1426–1435.

Fowler, M., 2009. Technical debt quadrant. URL https://www.martinfowler.com/bliki/TechnicalDebtQuadrant.html. (Accessed 13 June 2024).

GHArchive, 2024. GH Archive: Recording the public GitHub timeline. https://www.gharchive.org/. (Accessed 19 June 2024).

Guo, Y., Wan, Z., Xia, X., Lo, D., Yang, X., 2021. An empirical study of refactorings and technical debt in machine learning systems. In: 2021 IEEE/ACM 43rd International Conference on Software Engineering. ICSE, IEEE, pp. 238–250.

He, P., Liu, X., Gao, J., Chen, W., 2021. Deberta: Decoding-enhanced bert with disentangled attention. In: International Conference on Learning Representations. URL https://openreview.net/forum?id=XPZIaotutsD.

Holvitie, J., Licorish, S.A., Spínola, R.O., Hyrynsalmi, S., MacDonell, S.G., Mendes, T.S., Buchan, J., Leppänen, V., 2018. Technical debt and agile software development practices and processes: An industry practitioner survey. Inf. Softw. Technol. 96, 141–160.

Hu, Y., Khan, L., 2021. Uncertainty-aware reliable text classification. In: Proceedings of the 27th ACM SIGKDD Conference on Knowledge Discovery & Data Mining. pp. 628–636.

Huang, Q., Shihab, E., Xia, X., Lo, D., Li, S., 2018. SATD detector: a text-mining-based self-admitted technical debt detection tool. In: Proceedings of the 40th International Conference on Software Engineering: Companion Proceeedings. ACM, pp. 9–12.

Huang, J., Zhao, K., Liu, J., Liu, X., Xu, Z., Wang, X., 2022. Exploiting gated graph neural network for detecting and explaining self-admitted technical debts. J. Syst. Softw. 186, 111220.

Kacprzak, K., 2024. RoBERTa vs GPT-4: A comparative analysis of language model capabilities. Blog Post URL https://dsstream.com/roberta-vs-gpt-4-a-comparative-analysis-of-language-model-capabilities/.

Khan, J.Y., Uddin, G., 2022. Automatic detection and analysis of technical debts in peer-review documentation of r packages. In: 2022 IEEE International Conference on Software Analysis, Evolution and Reengineering. SANER, IEEE, pp. 765–776.

Lefever, J., Cai, Y., Cervantes, H., Kazman, R., Fang, H., 2021. On the lack of consensus among technical debt detection tools. In: 2021 IEEE/ACM 43rd International Conference on Software Engineering: Software Engineering in Practice. ICSE-SEIP, IEEE, pp. 121–130.

Lenarduzzi, V., Besker, T., Taibi, D., Martini, A., Fontana, F.A., 2021. A systematic literature review on technical debt prioritization: Strategies, processes, factors, and tools. J. Syst. Softw. 171, 110827.

Li, Y., Soliman, M., Avgeriou, P., 2023. Automatic identification of self-admitted technical debt from four different sources. Empir. Softw. Eng. 28 (3), 65.

Liu, Z., Huang, Q., Xia, X., Shihab, E., Lo, D., Li, S., 2018. SATD detector: A text-mining-based self-admitted technical debt detection tool. In: 2018 IEEE/ ACM 40th International Conference on Software Engineering: Companion. ICSE- Companion, pp. 9–12, URL https://api.semanticscholar.org/CorpusID:49294647.

Liu, Z., Huang, Q., Xia, X., Shihab, E., Lo, D., Li, S., 2021. DebtHunter: A machine learning-based approach for detecting self-admitted technical debt. In: Proceedings of the 29th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering. ACM, pp. 1262–1273.

Maldonado, E.d.S., Shihab, E., Tsantalis, N., 2017. Using natural language processing to automatically detect self-admitted technical debt. IEEE Trans. Softw. Eng. 43 (11), 1044–1062.

Microsoft, 2024. Visual Studio Code. https://github.com/microsoft/vscode. (Accessed 28 June 2024).

Montgomery, L., Lüders, C., Maalej, W., 2022. An alternative issue tracking dataset of public jira repositories. In: Proceedings of the 19th International Conference on Mining Software Repositories. pp. 73–77.

von der Mosel, J., Trautsch, A., Herbold, S., 2023. On the validity of pre-trained transformers for natural language processing in the software engineering domain. IEEE Trans. Softw. Eng. 49 (4), 1487–1507. http://dx.doi.org/10.1109/TSE.2022.3178469.

Onnx, 2024. ONNX: Open neural network exchange. https://onnx.ai/. (Accessed 24 June 2024).

Ovadia, Y., Fertig, E., Ren, J., Nado, Z., Sculley, D., Nowozin, S., Dillon, J., Lakshminarayanan, B., Snoek, J., 2019. Can you trust your model's uncertainty? Evaluating predictive uncertainty under dataset shift. Adv. Neural Inf. Process. Syst. 32.

Pinna, A., Lunesu, M.I., Orrù, S., Tonelli, R., 2023. Investigation on self-admitted technical debt in open-source blockchain projects. Futur. Internet 15, 232, URL https://api.semanticscholar.org/CorpusID:259758199.

Radford, A., Wu, J., Child, R., Luan, D., Amodei, D., Sutskever, I., et al., 2019. Language models are unsupervised multitask learners. OpenAI Blog 1 (8), 9.

Raj, C., Agarwal, A., Bharathy, G., Narayan, B., Prasad, M., 2021. Cyberbullying detection: Hybrid models based on machine learning and natural language processing techniques. Electronics 10 (22), 2810.

Recupito, G., Pecorelli, F., Catolino, G., Lenarduzzi, V., Taibi, D., Di Nucci, D., Palomba, F., 2024. Technical debt in AI-enabled systems: On the prevalence, severity, impact, and management strategies for code and architecture. J. Syst. Softw. 216, 112151.

Ren, X., Xing, Z., Xia, X., Lo, D., Grundy, J., 2022a. DebtFree: minimizing labeling cost in self-admitted technical debt identification using semi-supervised learning. Empir. Softw. Eng. 27 (4), 1–38.

Ren, X., Xing, Z., Xia, X., Lo, D., Grundy, J., 2022b. Identifying self-admitted technical debt in issue tracking systems using machine learning. Empir. Softw. Eng. 27 (4), 1–37.

Ren, X., Xing, Z., Xia, X., Lo, D., Wang, X., Grundy, J., 2019. Neural network-based detection of self-admitted technical debt: From performance to explainability. ACM Trans. Softw. Eng. Methodol. 28 (3), 1–45.

Sanh, V., Debut, L., Chaumond, J., Wolf, T., 2019. Distilbert, a distilled version of BERT: smaller, faster, cheaper and lighter. ArXiv abs/1910.01108.

Santos, E.P., Gomes, F., Freire, S., Mendonça, M., Mendes, T.S., Spínola, R., 2022. Technical debt on agile projects: Managers' point of view at stack exchange. In: Proceedings of the XXI Brazilian Symposium on Software Quality. pp. 1–9.

Sas, D., Avgeriou, P., 2023. An architectural technical debt index based on machine learning and architectural smells. IEEE Trans. Softw. Eng. 49 (8), 4169–4195. http://dx.doi.org/10.1109/TSE.2023.3286179.

Sculley, D., Holt, G., Golovin, D., Davydov, E., Phillips, T., Ebner, D., Chaudhary, V., Young, M., Crespo, J.-F., Dennison, D., 2015. Hidden technical debt in machine learning systems. In: Advances in Neural Information Processing Systems. pp. 2503–2511.

Simon, E.I.-O., Hettiarachchi, C., Potanin, A., Suominen, H., Fard, F., 2024. Automated detection of algorithm debt in deep learning frameworks: An empirical study. arXiv preprint arXiv:2408.10529.

Skryseth, D., Shivashankar, K., Pilán, I., Martini, A., 2023. Technical debt classification in issue trackers using natural language processing based on transformers. In: 2023 ACM/IEEE International Conference on Technical Debt (TechDebt). pp. 92–101. http://dx.doi.org/10.1109/TechDebt59074.2023.00017.

Spínola, R.O., Zazworka, N., Vetro, A., Shull, F., Seaman, C., 2019. Understanding automated and human-based technical debt identification approaches-a two-phase study. J. Braz. Comput. Soc. 25 (1), 5. http://dx.doi.org/10.1186/s13173-019-0087-5.

Sridharan, M., Mantyla, M., Rantala, L., Claes, M., 2021. Data balancing improves self-admitted technical debt detection. In: 2021 IEEE/ACM 18th International Conference on Mining Software Repositories. MSR, IEEE, pp. 358–368.

Student, 1908. The probable error of a mean. Biometrika 1–25.

Sundjaja, J., Shrestha, R., Krishan, K., 2023. Mcnemar and mann-whitney u tests. StatPearls.

Syed, K., Sleeman IV, W., Ivey, K., Hagan, M., Palta, J., Kapoor, R., Ghosh, P., 2020. Integrated natural language processing and machine learning models for standardizing radiotherapy structure names. In: Healthcare. Vol. 8, MDPI, p. 120.

Szeghalmy, S., Fazekas, A., 2023. A comparative study of the use of stratified cross-validation and distribution-balanced stratified cross-validation in imbalanced learning. Sensors 23 (4), 2333.

Tan, J., Li, X., Xia, X., Zhang, Z., Huang, X., Huang, T., 2021. Ensemble learning for self-admitted technical debt detection. In: 2021 IEEE International Conference on Software Analysis, Evolution and Reengineering. SANER, IEEE, pp. 610–614.

Tsoukalas, D., Mittas, N., Arvanitou, E.-M., Ampatzoglou, A., Chatzigeorgiou, A., Kechagias, D., 2024. Local and global explainability for technical debt identification. IEEE Trans. Softw. Eng..

Tsoukalas, D., Siavvas, M., Jankovic, M., Kehagias, D., Chatzigeorgiou, A., Tzovaras, D., 2018. Methods and tools for TD estimation and forecasting: A state-of-the-art survey. In: 2018 International Conference on Intelligent Systems. IS, IEEE, pp. 698–705.

Vargha, A., Delaney, H.D., 2000. A critique and improvement of the CL common language effect size statistics of McGraw and wong. J. Educ. Behav. Stat. 25 (2), 101–132.

Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A.N., Kaiser, L.u., Polosukhin, I., 2017. Attention is all you need. In: Guyon, I., Luxburg, U.V., Bengio, S., Wallach, H., Fergus, R., Vishwanathan, S., Garnett, R. (Eds.), Advances in Neural Information Processing Systems. Vol. 30, Curran Associates, Inc..

Wolf, T., Debut, L., Sanh, V., Chaumond, J., Delangue, C., Moi, A., Cistac, P., Rault, T., Louf, R., Funtowicz, M., et al., 2019. Huggingface's transformers: State-of-the-art natural language processing. arXiv preprint arXiv:1910.03771.

Younes, Y., Mathiak, B., 2022. Handling class imbalance when detecting dataset mentions with pre-trained language models. In: Proceedings of the 5th International Conference on Natural Language and Speech Processing. ICNLSP 2022, pp. 79–88.

Zhuang, L., Wayne, L., Ya, S., Jun, Z., 2021. A robustly optimized BERT pre-training approach with post-training. In: Li, S., Sun, M., Liu, Y., Wu, H., Liu, K., Che, W., He, S., Rao, G. (Eds.), Proceedings of the 20th Chinese National Conference on Computational Linguistics. Chinese Information Processing Society of China, Huhhot, China, pp. 1218–1227, URL https://aclanthology.org/2021.ccl-1.108.